# CUSTOMER BEHAVIOR AND CHURN EDA INSIGHTS

## Understanding Customer Behavior, Risk & Retention

# Table of Content


- Introduction
- Business problems
- Churn Definition
- Objective of analysis
- About the dataset
- Import Libraries
- Data Loading & Initial Inspection
- Data Cleaning & Preprocessing

# INTRODUCTION

Customer churn is a critical challenge for subscription-based businesses, as losing customers directly affects revenue, profitability, and long-term growth. Understanding customer behavior and identifying the key factors that lead to churn are essential for developing effective retention strategies.

This project performs Exploratory Data Analysis (EDA) on a customer behavior and churn dataset to identify patterns, trends, and risk factors associated with customer churn.

The main purposes of this analysis are:

- To understand customer demographics, subscription behavior, and engagement patterns
- To identify key factors that contribute to customer churn
- To analyze the relationship between customer satisfaction, usage behavior, and retention
- To detect early warning signals such as inactivity and low engagement
- To segment customers based on churn risk levels
- To generate meaningful business insights and data-driven recommendations

This analysis helps businesses improve customer retention, enhance customer experience, and make informed strategic decisions to reduce churn and increase long-term customer value.

# BUSINESS PROBLEMS

Customer churn is a significant challenge for subscription-based businesses, as customer loss directly reduces revenue and long-term profitability. Since retaining existing customers is generally more cost-effective than acquiring new ones, improving customer retention is a critical business objective.

The business aims to understand:

- Which customers are most likely to churn
- What behavioral and service-related factors drive churn
- How engagement, satisfaction, and customer support interactions influence retention
- How to detect high-risk customers early and take preventive action
- How to leverage data-driven insights to strengthen retention strategies

By identifying the key drivers of churn, businesses can implement targeted interventions to reduce customer loss and increase customer lifetime value.

# CHURN DEFINITION

In this dataset, churn refers to a customer discontinuing or canceling the subscription service.

The target variable **churn** is defined as:
- **0** = Retained Customer (Customer continues using the service)
- **1** = Churned Customer (Customer has stopped using the service)

Customer churn is influenced by multiple behavioral and service-related factors, including:

- Low customer satisfaction scores
- Decreased usage frequency and engagement levels
- Longer inactivity periods (higher days since last login)
- Increased number of support tickets
- Limited response to promotions and discounts

Understanding these factors enables businesses to identify at-risk customers and design effective retention strategies.

# OBJECTIVE OF ANALYSIS

The primary objective of this analysis is to examine customer behavior and determine the key factors that drive customer churn in a subscription-based service.

The specific objectives include:

- To analyze customer demographics, subscription details, and billing characteristics
- To evaluate customer engagement patterns such as usage frequency and session activity
- To identify behavioral and service-related factors contributing to churn
- To detect early warning indicators, including inactivity and low satisfaction levels
- To examine the relationship between customer satisfaction, support interactions, and retention
- To perform churn risk segmentation to identify high-risk customers
- To generate actionable business insights and strategic recommendations

This analysis enables businesses to make data-driven decisions aimed at reducing churn, improving retention rates, and increasing long-term customer value.

# ABOUT THE DATASET

- Customer Behavior and Churn Simulation Dataset (15,000+ customer records)
- Exploratory Data Analysis (EDA) on customer behavior and churn patterns
- The dataset was accessed in February 2026
- The Data Set was downloaded from Kaggle, from the following link

# IMPORT LIBRARIES

- **pandas :** For data cleaning, manipulation, and analysis.
- **NumPy :** For numerical operations and array handling.
- **matplotlib :** For creating basic static visualizations.
- **seaborn :** For advanced and attractive statistical plots.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

# Data Loading & Initial Inspection

- Load the csv file with the pandas
- creating the dataframe and understanding the data present in the dataset using pandas

```
df = pd.read_csv(r"C:\Users\DELL\Desktop\new project\Customer Behavior & Churn Simulation Dataset\synthetic_customer_behavior_and_churn.csv")
```

- Viewing the first five Rows of the data

```
df.head(5)
```

| | customer_id | signup_date | tenure_months | age | gender | region | income_level | subscription_type | monthly_charges | total_charges | usage_frequency | avg_session_dura |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | CUST_000001 | 2022-08-17 | 28 | 47 | Male | North America | low | basic | 35.22 | 1034.06 | medium | |
| 1 | CUST_000002 | 2022-02-21 | 34 | 40 | Male | Europe | low | standard | 63.56 | 2065.11 | medium | |
| 2 | CUST_000003 | 2023-07-18 | 17 | 49 | Female | Asia Pacific | low | standard | 74.37 | 1248.11 | low | |
| 3 | CUST_000004 | 2023-05-17 | 19 | 60 | Female | Latin America | medium | premium | 98.28 | 1540.50 | high | |
| 4 | CUST_000005 | 2023-04-03 | 20 | 39 | Female | Asia Pacific | medium | standard | 61.47 | 1166.17 | low | |

## Display all columns and row

**Note** : Some the output of notebook does not present the complete output, therefore we can increase the limit of columns view and row view by using these commands:

```
#to display all row and columns in the dataframe
pd.set_option('display.max_columns',None)
pd.set_option('display.max_rows',None)
```

## Hide all warnings runtime

```
#to ignore unnecessary warning messages
import warnings
warnings.filterwarnings('ignore')
```

- Let's check the shape of the dataset to see how many rows and columns it contains.

```
#to check the shape of the dataset
print(f"No. of columns in this dataset = {df.shape[0]}")
print(f"No. of rows in this dataset = {df.shape[1]}")
```

```
No. of columns in this dataset = 15000
No. of rows in this dataset = 21
```

- Let's view the exact column names in the **Customer Behavior & Churn Simulation Dataset**, which we might need to reference or reuse later

```
#to view exact columns names
df.columns
```

```
Index(['customer_id', 'signup_date', 'tenure_months', 'age', 'gender',
       'region', 'income_level', 'subscription_type', 'monthly_charges',
       'total_charges', 'usage_frequency', 'avg_session_duration_minutes',
       'number_of_logins_per_month', 'number_of_support_tickets',
       'satisfaction_score', 'payment_method', 'contract_type',
       'last_login_days_ago', 'promotional_response', 'discount_used',
       'churn'],
      dtype='object')
```

- Displaying a random sample of 5 rows to understand the variety in the dataset.

```
#to see random sample of 5 rows to undersatnd the variety
df.sample(5)
```

| | customer_id | signup_date | tenure_months | age | gender | region | income_level | subscription_type | monthly_charges | total_charges | usage_frequency | avg_session_ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **11486** | CUST_011487 | 2024-12-05 | 1 | 18 | Non-binary | Europe | medium | standard | 78.06 | 78.61 | high | |
| **1853** | CUST_001854 | 2023-02-22 | 22 | 53 | Non-binary | North America | low | basic | 33.79 | 684.94 | low | |
| **2021** | CUST_002022 | 2023-02-19 | 22 | 45 | Male | Asia Pacific | medium | standard | 73.17 | 1542.69 | medium | |
| **2999** | CUST_003000 | 2022-01-25 | 35 | 40 | Female | Europe | low | basic | 37.72 | 1404.50 | low | |
| **1882** | CUST_001883 | 2023-10-12 | 14 | 27 | Male | Latin America | low | basic | 22.42 | 338.85 | low | |

# Data Cleaning & Preprocessing

# Handle Missing Values

```
df.shape
```

```
(15000, 24)
```

- Let's take a look at the missing values in the dataset to understand data quality.

```
#to check any missing values
df.isnull().sum()
```

```
customer_id                     0
signup_date                     0
tenure_months                   0
age                             0
gender                          0
region                          0
income_level                    0
subscription_type               0
monthly_charges                 0
total_charges                   0
usage_frequency                 0
avg_session_duration_minutes    0
number_of_logins_per_month      0
number_of_support_tickets       0
satisfaction_score              0
payment_method                  0
contract_type                   0
last_login_days_ago             0
promotional_response            0
discount_used                   0
churn                           0
dtype: int64
```

# Handle Duplicate

- Let's begin by checking if there are any completely duplicated rows in the dataset.

```
#to check duplicate rows
df.duplicated().sum()
```

```
0
```

- Let's investigate which columns contain repeated values to identify potential redundancy or repeated patterns

```
#loop through each column to find howmanytimes values are repeated
for col in df.columns :
    print(f"column:{col} = Duplicates: {df[col].duplicated().sum()}")
```

```
column:customer_id = Duplicates: 0
column:signup_date = Duplicates: 13904
column:tenure_months = Duplicates: 14965
column:age = Duplicates: 14937
column:gender = Duplicates: 14997
column:region = Duplicates: 14995
column:income_level = Duplicates: 14997
column:subscription_type = Duplicates: 14997
column:monthly_charges = Duplicates: 8443
column:total_charges = Duplicates: 497
column:usage_frequency = Duplicates: 14997
column:avg_session_duration_minutes = Duplicates: 13851
column:number_of_logins_per_month = Duplicates: 14951
column:number_of_support_tickets = Duplicates: 14988
column:satisfaction_score = Duplicates: 14995
column:payment_method = Duplicates: 14996
column:contract_type = Duplicates: 14998
column:last_login_days_ago = Duplicates: 14910
column:promotional_response = Duplicates: 14998
column:discount_used = Duplicates: 14998
column:churn = Duplicates: 14998
```

- Let’s check data types of each columns

```
#to check data types of each column
print("\ndata types of each column:")
print(df.dtypes)
```

```
data types of each column:
customer_id                     object
signup_date                     object
tenure_months                    int64
age                              int64
gender                          object
region                          object
income_level                    object
subscription_type               object
monthly_charges                float64
total_charges                  float64
usage_frequency                 object
avg_session_duration_minutes   float64
number_of_logins_per_month       int64
number_of_support_tickets        int64
satisfaction_score               int64
payment_method                  object
contract_type                   object
last_login_days_ago              int64
promotional_response            object
discount_used                   object
churn                            int64
dtype: object
```

- Let’s identify categorical and constant columns

```
#to identify categorical and constant columns
unique_values = df.nunique()

print("\nUnique values count in each column:")
print(unique_values)
```

```
Unique values count in each column:
customer_id                     15000
signup_date                      1096
tenure_months                      35
age                                63
gender                              3
region                              5
income_level                        3
subscription_type                   3
monthly_charges                  6557
total_charges                   14503
usage_frequency                     3
avg_session_duration_minutes     1149
number_of_logins_per_month         49
number_of_support_tickets          12
satisfaction_score                  5
payment_method                      4
contract_type                       2
last_login_days_ago                90
promotional_response                2
discount_used                       2
churn                               2
dtype: int64
```

# Statistical Summary of Numerical Columns

```python
# to show mean,min,max,std etc
print("\nStatistical summary of numerical columns:")
print(df.describe())
```

```
Statistical summary of numerical columns:
       tenure_months           age  monthly_charges  total_charges  \
count   15000.000000  15000.000000     15000.000000   15000.000000
mean       17.448467     41.646933        59.046174     978.208528
std        10.304214     11.740994        32.996490     864.474724
min         1.000000     18.000000        19.990000      16.500000
25%         8.000000     33.000000        31.640000     351.145000
50%        17.000000     42.000000        55.150000     735.260000
75%        26.000000     50.000000        73.742500    1372.332500
max        35.000000     80.000000       149.950000    5506.350000

       avg_session_duration_minutes  number_of_logins_per_month  \
count                  15000.000000                15000.000000
mean                      45.235633                   18.519600
std                       29.244734                   12.335911
min                        5.000000                    1.000000
25%                       21.900000                    9.000000
50%                       40.500000                   16.000000
75%                       58.500000                   24.000000
max                      119.900000                   49.000000

       number_of_support_tickets  satisfaction_score  last_login_days_ago  \
count               15000.000000         15000.00000         15000.000000
mean                    2.927133             3.38440            21.748533
std                     2.609012             1.24951            22.282570
min                     0.000000             1.00000             0.000000
25%                     1.000000             3.00000             5.000000
50%                     2.000000             4.00000            13.000000
75%                     4.000000             4.00000            32.000000
max                    11.000000             5.00000            89.000000

              churn
count  15000.000000
mean       0.318867
std        0.466053
min        0.000000
25%        0.000000
50%        0.000000
75%        1.000000
max        1.000000
```

# Target Variable (Churn) Analysis

The target variable is "churn"
churn = 0 ⟶ Customer retained churn = 1 ⟶ Customer churned

This section analyzes:

1. Count of churn vs retained customers

2. Percentage distribution

3. Visualization using count plot and pie chart

- Let’s confirms churn contains valid binary values

```
#to comfirms churn contains valid binary values
print("Unique values in churn column :")
print(df["churn"].unique())
```

```
Unique values in churn column :
[0 1]
```

- Customer’s count in each churn category

```
#this counts customers in each churn category
churn_counts=df["churn"].value_counts()
print("\nChurn counts:")
print(churn_counts)
```

```
Churn counts:
churn
0    10217
1     4783
Name: count, dtype: int64
```

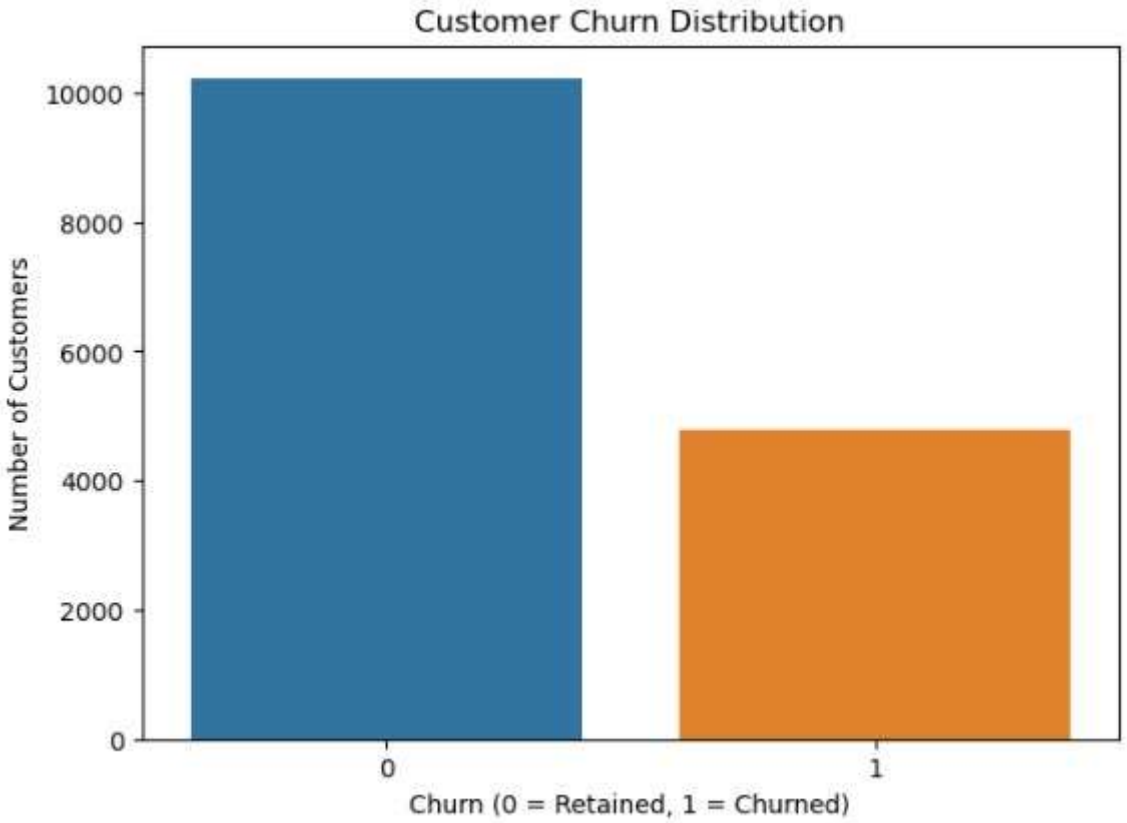
- Let’s calculate percentage of each class

```
#this counts customers in each churn category
churn_counts=df["churn"].value_counts()
print("\nChurn counts:")
print(churn_counts)
```

```
Churn counts:
churn
0    10217
1     4783
Name: count, dtype: int64
```

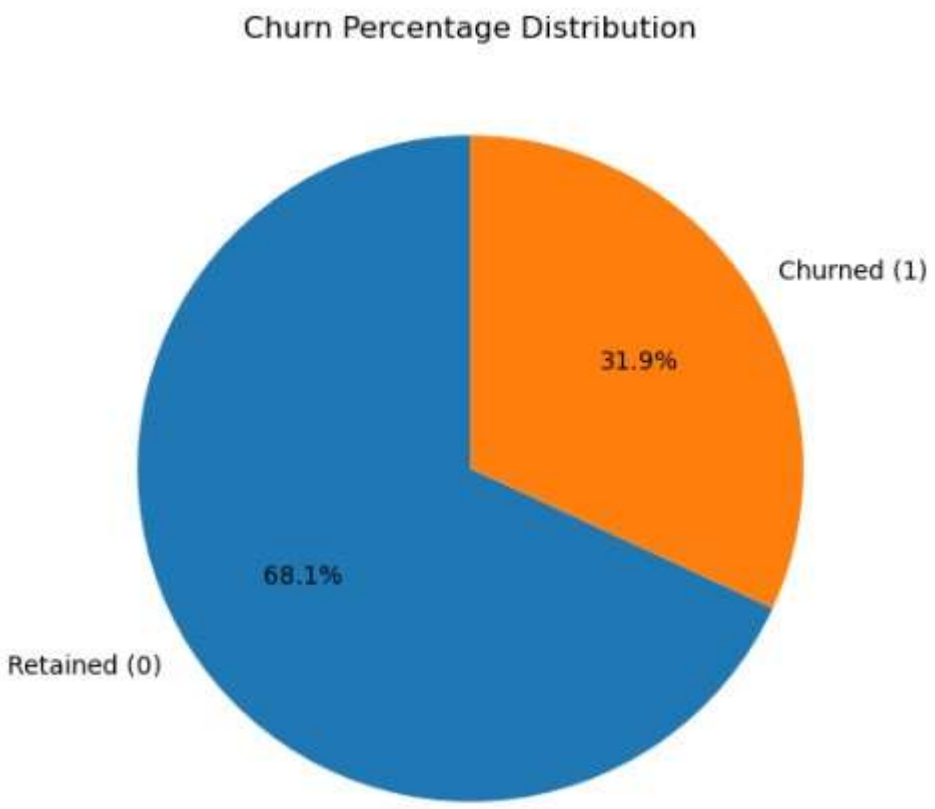
- Let’s visualizes number of retained vs churned customers

```
# This visualizes number of retained vs churned customers

plt.figure(figsize=(7,5))

# Create countplot
sns.countplot(x="churn", data=df)

# Add title and labels
plt.title("Customer Churn Distribution")
plt.xlabel("Churn (0 = Retained, 1 = Churned)")
plt.ylabel("Number of Customers")

# Show plot
plt.show()
```



- Let’s visualizes churn proportion using pie chart

```
# This shows churn proportion visually

plt.figure(figsize=(6,6))

# Create churn counts
churn_counts = df["churn"].value_counts()

# Create pie chart
plt.pie(
    churn_counts,
    labels=["Retained (0)", "Churned (1)"],
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Churn Percentage Distribution")
plt.show()
```



- Churn Summary

```python
# This creates a summary table for business understanding

# Create churn counts (sorted to keep 0 then 1)
churn_counts = df["churn"].value_counts().sort_index()

# Calculate percentage
churn_percentage = (churn_counts / churn_counts.sum()) * 100

# Create summary table
churn_summary = pd.DataFrame({
    "Customer Count": churn_counts,
    "Percentage (%)": churn_percentage.round(2)
})

print("\nChurn Summary Table:")
print(churn_summary)
```

```
Churn Summary Table:
       Customer Count  Percentage (%)
churn
0               10217           68.11
1                4783           31.89
```

# Feature Engineering

- Feature Engineering involves creating meaningful new variables from existing data to enhance analysis.
- In this dataset, new features are derived from **signup_date** and **spending behavior**.
- These features help better understand customer lifecycle, engagement patterns, and overall customer value.

- Creating Customer Tenure Features from Signup Date

```python
#convert Signup_date to Datetime
df["signup_date"] = pd.to_datetime(df["signup_date"],errors= "coerce")

#create tenure feature
current_date = pd.to_datetime("today")
df["tenure_days"] = (current_date - df["signup_date"]).dt.days

#Create tenure in Months
df["tenure_months"]=df["tenure_days"] // 30

#Verify new features
print(df[["signup_date","tenure_days","tenure_months"]].head())
```

| | signup_date | tenure_days | tenure_months |
|---|---|---|---|
| 0 | 2022-08-17 | 1289 | 42 |
| 1 | 2022-02-21 | 1466 | 48 |
| 2 | 2023-07-18 | 954 | 31 |
| 3 | 2023-05-17 | 1016 | 33 |
| 4 | 2023-04-03 | 1060 | 35 |

- How long customer using service ?

```
#Tenure shows how long customer using service

#Get today's date
current_date=pd.to_datetime("today")

#Calculate tenure in days
df["tenure_days"]=(current_date - df["signup_date"]).dt.days

#show sample values
print("Sample tenure_days:")
print(df["tenure_days"].head())
```

```
Sample tenure_days:
0    1289
1    1466
2     954
3    1016
4    1060
Name: tenure_days, dtype: int64
```

- Converting tenure from days to months

```
#for easy business understanding converts tenure from days to months

df["tenure_months"] = df["tenure_days"] // 30

#showsample values
print("\nSample tenure_months:")
print(df["tenure_months"].head())
```

```
Sample tenure_months:
0    42
1    48
2    31
3    33
4    35
Name: tenure_months, dtype: int64
```

- Converting important columns to numeric

```
#Convert important columns to numeric

numeric_columns= [
    "total_charges",
    "usage_frequency",
    "avg_session_duration_minutes",
    "tenure_months",
    "last_login_days_ago",
    "number_of_support_tickets",
    "monthly_charges"
]

for col in numeric_columns:
    df[col]=pd.to_numeric(df[col], errors = "coerce")
print("Columns converted to numeric successfully")
```

```
Columns converted to numeric successfully
```

- Creating Spending, Engagement, and Inactivity Features

```python
#Feature Engineering

#average monthly spend
df["avg_monthly_spend"] = df["total_charges"] / (df["tenure_months"] + 1)

#Usage intensity
df["usage_intensity"] = (
    df["usage_frequency"] * df["avg_session_duration_minutes"]
)


#Inactive customer flag
df["inactive_customer"]=(
    df["last_login_days_ago"] > df["last_login_days_ago"].median()
)


#verify
print(df[["avg_monthly_spend",
         "usage_intensity",
         "inactive_customer"]].head())
```

| | avg_monthly_spend | usage_intensity | inactive_customer |
|---|---|---|---|
| 0 | 24.047907 | NaN | False |
| 1 | 42.145102 | NaN | False |
| 2 | 39.003437 | NaN | True |
| 3 | 45.308824 | NaN | False |
| 4 | 32.393611 | NaN | True |

- Creating Average Monthly Spend Feature

```
#create avg_monthly_spend feature

#Convert to numeric first
df["total_charges"]= pd.to_numeric(df["total_charges"], errors = "coerce")
df["tenure_months"]= pd.to_numeric(df["tenure_months"], errors = "coerce")

#create average monthly spend feature
df["avg_monthly_spend"] = df["total_charges"] / (df["tenure_months"]+1)

#Show result
print(df[["total_charges","tenure_months","avg_monthly_spend"]].head())
```

| | total_charges | tenure_months | avg_monthly_spend |
|---|---|---|---|
| 0 | 1034.06 | 42 | 24.047907 |
| 1 | 2065.11 | 48 | 42.145102 |
| 2 | 1248.11 | 31 | 39.003437 |
| 3 | 1540.50 | 33 | 45.308824 |
| 4 | 1166.17 | 35 | 32.393611 |

- Creating Usage Intensity Feature

```
#create usage_intensity feature


#Convert columns to numeric first

df["usage_frequency"] = pd.to_numeric(df["usage_frequency"],errors ="coerce")
df["avg_session_duration_minutes"] = pd.to_numeric (df["avg_session_duration_minutes"], errors = "coerce"
)


#create usage intensity feature

df["usage_intensity"] = (
    df["usage_frequency"]*df["avg_session_duration_minutes"]
)

#show result
print(df[["usage_frequency","avg_session_duration_minutes","usage_intensity"]].head())
```

| | usage_frequency | avg_session_duration_minutes | usage_intensity |
|---|---|---|---|
| 0 | NaN | 55.3 | NaN |
| 1 | NaN | 25.7 | NaN |
| 2 | NaN | 9.0 | NaN |
| 3 | NaN | 99.4 | NaN |
| 4 | NaN | 8.4 | NaN |

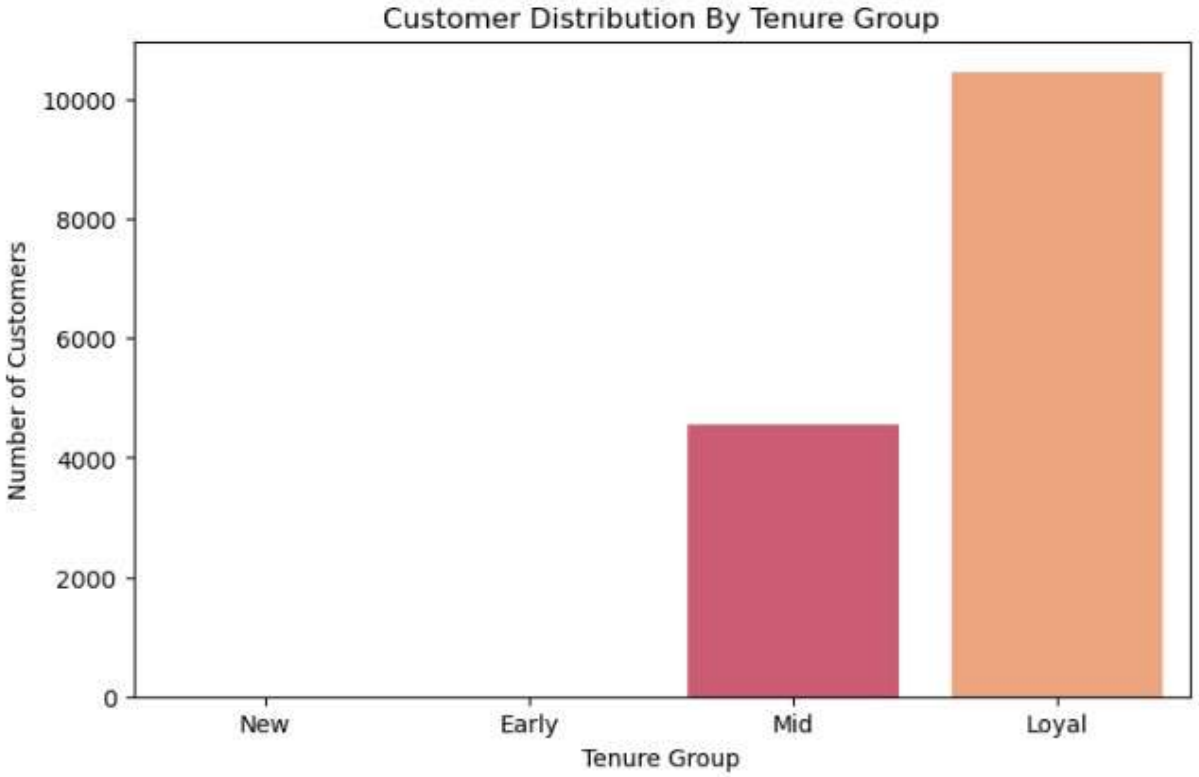
- Grouping Customers Into Lifecycle stages

```
#This groups customers into lifecycle stages
df["tenure_group"] =pd.cut(
    df["tenure_months"],
    bins=[0,6,12,24,60],
    labels=["New","Early","Mid","Loyal"]
)

print("\nSample tenure_group :")
print(df["tenure_group"].head())
```

```
Sample tenure_group :
0    Loyal
1    Loyal
2    Loyal
3    Loyal
4    Loyal
Name: tenure_group, dtype: category
Categories (4, object): ['New' < 'Early' < 'Mid' < 'Loyal']
```

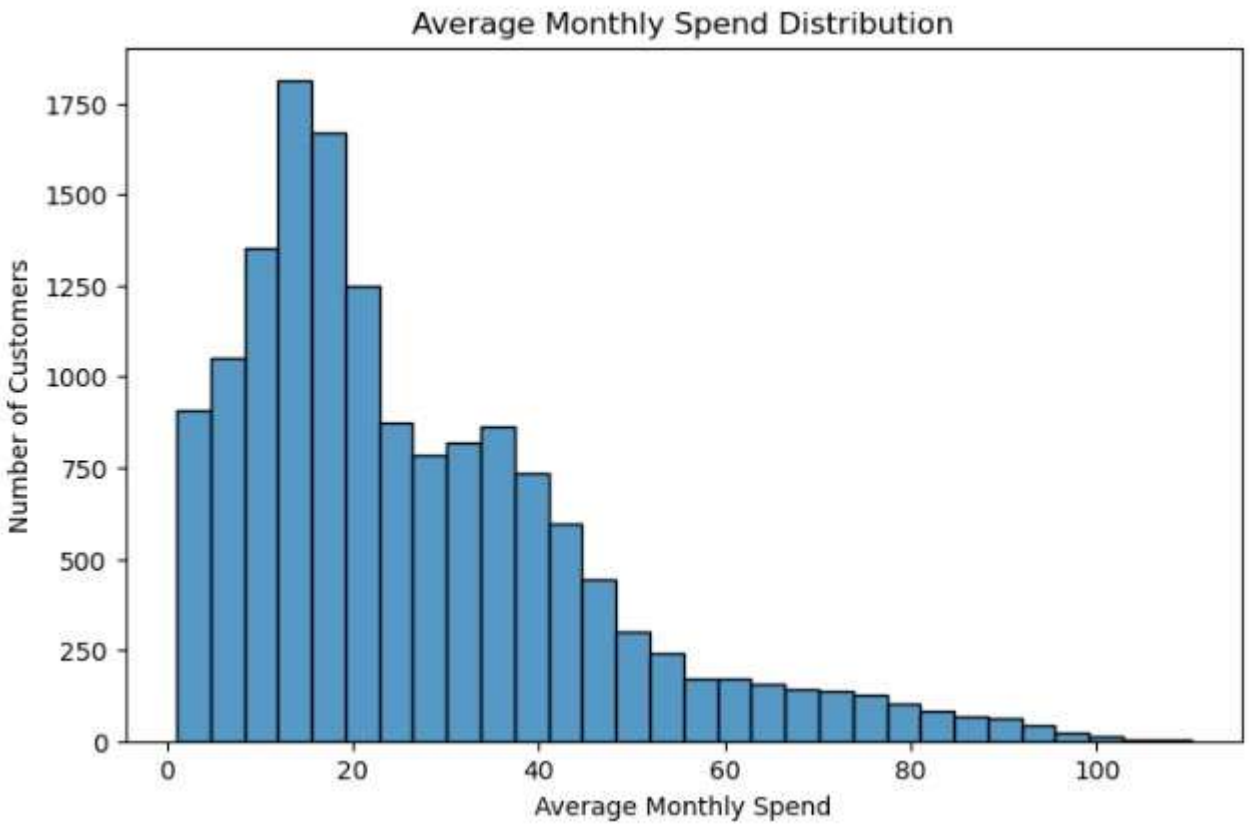
- Graph To Show Customers Distribution Across Life Cycle Stages

```
#to show how customers are distributed across life cycle stages


plt.figure(figsize=(8,5))

sns.countplot(x="tenure_group",data=df,palette="magma")

#add title and labels
plt.title("Customer Distribution By Tenure Group")
plt.xlabel("Tenure Group")
plt.ylabel("Number of Customers")

plt.show()
```



- Spending Behavior Distribution

```
#showing spending behavior distribution

plt.figure(figsize=(8,5))

sns.histplot(df["avg_monthly_spend"],bins=30)

#add labels and title
plt.title("Average Monthly Spend Distribution")
plt.xlabel("Average Monthly Spend")
plt.ylabel("Number of Customers")

plt.show()
```



- First 5 Rows of New Feature

```python
#show first 5 rowsof new features
new_features= [
    "tenure_days",
    "tenure_months",
    "avg_monthly_spend",
    "usage_intensity",
    "tenure_group"
]

print("\nNew Feature Sample:")
print(df[new_features].head())
```

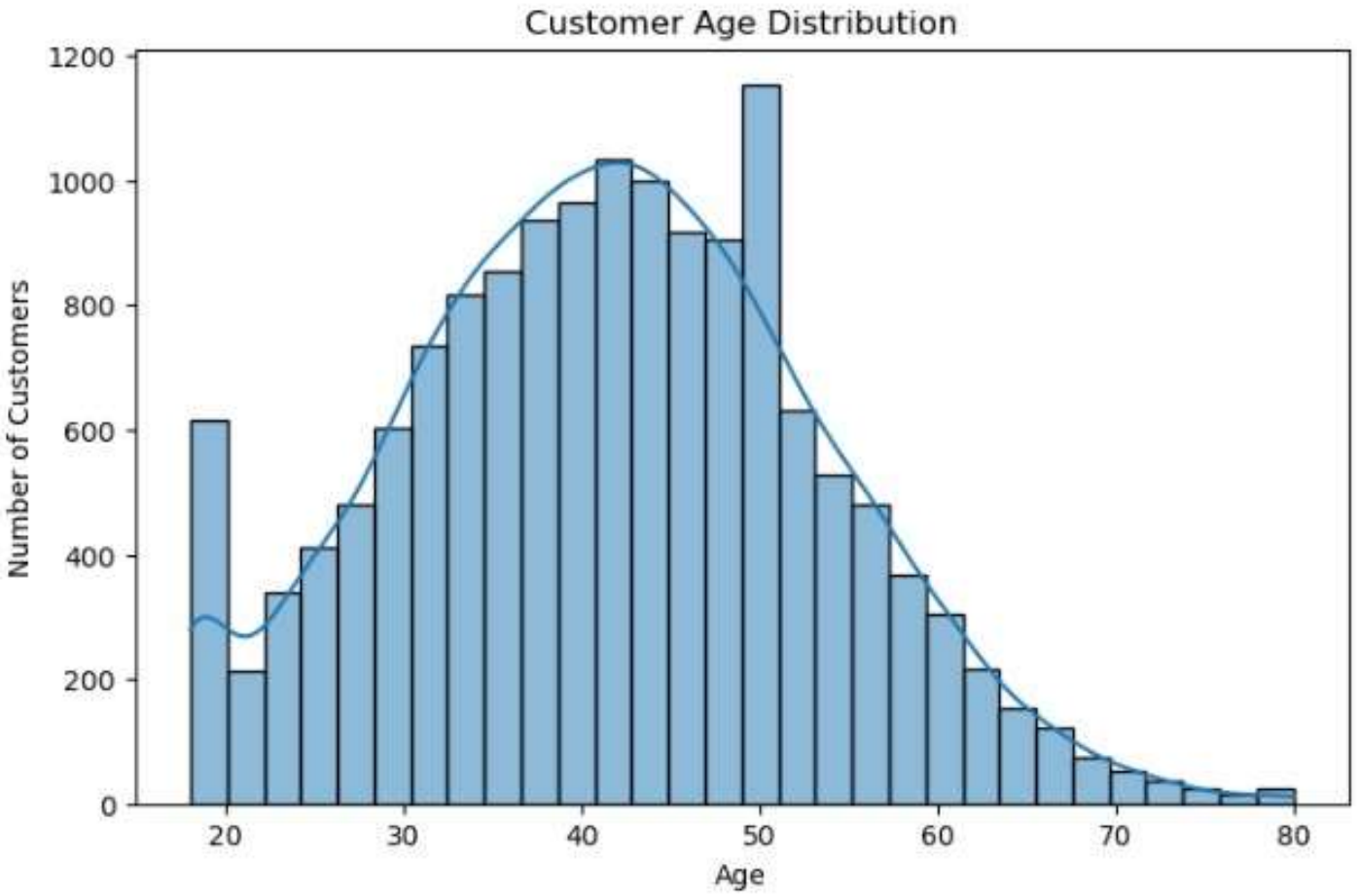
```
New Feature Sample:
   tenure_days  tenure_months  avg_monthly_spend  usage_intensity tenure_group
0         1289             42          24.047907              NaN        Loyal
1         1466             48          42.145102              NaN        Loyal
2          954             31          39.003437              NaN        Loyal
3         1016             33          45.308824              NaN        Loyal
4         1060             35          32.393611              NaN        Loyal
```

# Customer Demographics Analysis

- Analyze key customer demographic features: age, gender, region, and income level.
- Examine the distribution of these demographic variables.
- Evaluate their relationship and impact on customer churn.

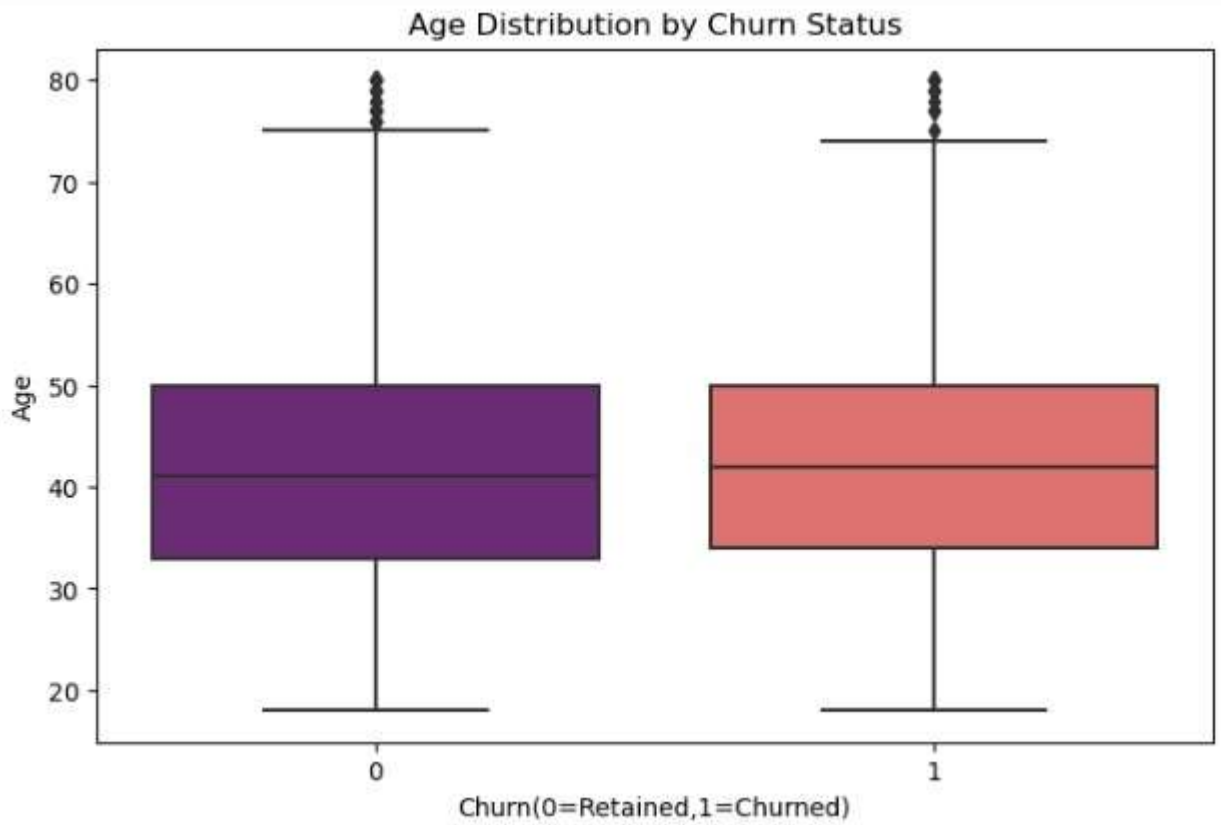
- Age Distribution Analysis

```
#shows how customer ages are destributed

plt.figure(figsize=(8,5))

#create histogram for age distribution
sns.histplot(df["age"], bins=30,kde=True)

#add titles and labels
plt.title("Customer Age Distribution")
plt.xlabel("Age")
plt.ylabel("Number of Customers")

plt.show()
```



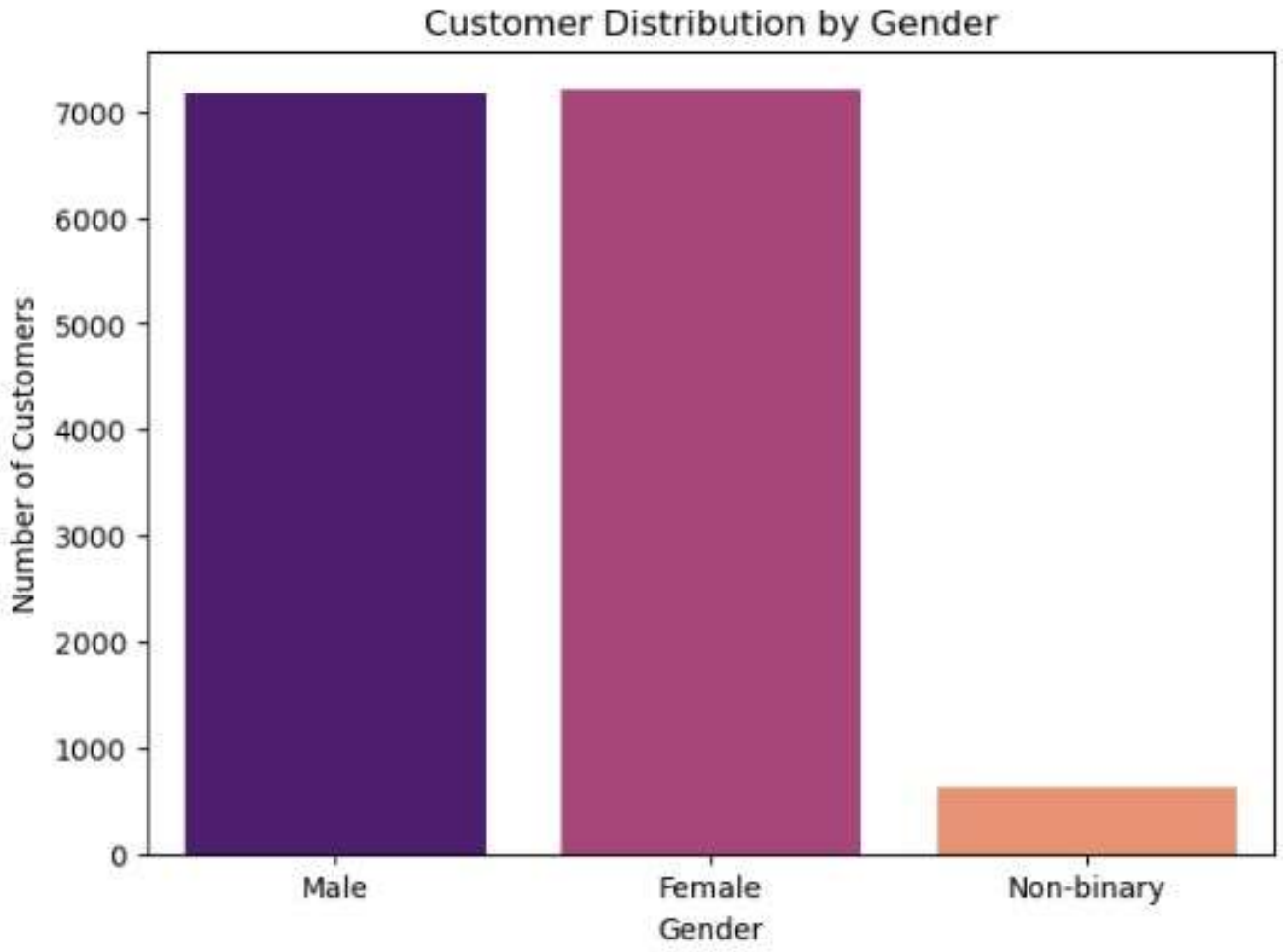
- Age vs churn analysis

```
#Thisshows whether churn differs by age

plt.figure(figsize=(8,5))

#Boxplot compares age distribution between churned and retained custome

sns.boxplot(x="churn",y="age",data=df,palette="magma")

#add title and labels

plt.title("Age Distribution by Churn Status")
plt.xlabel("Churn(0=Retained,1=Churned)")
plt.ylabel("Age")

plt.show()
```



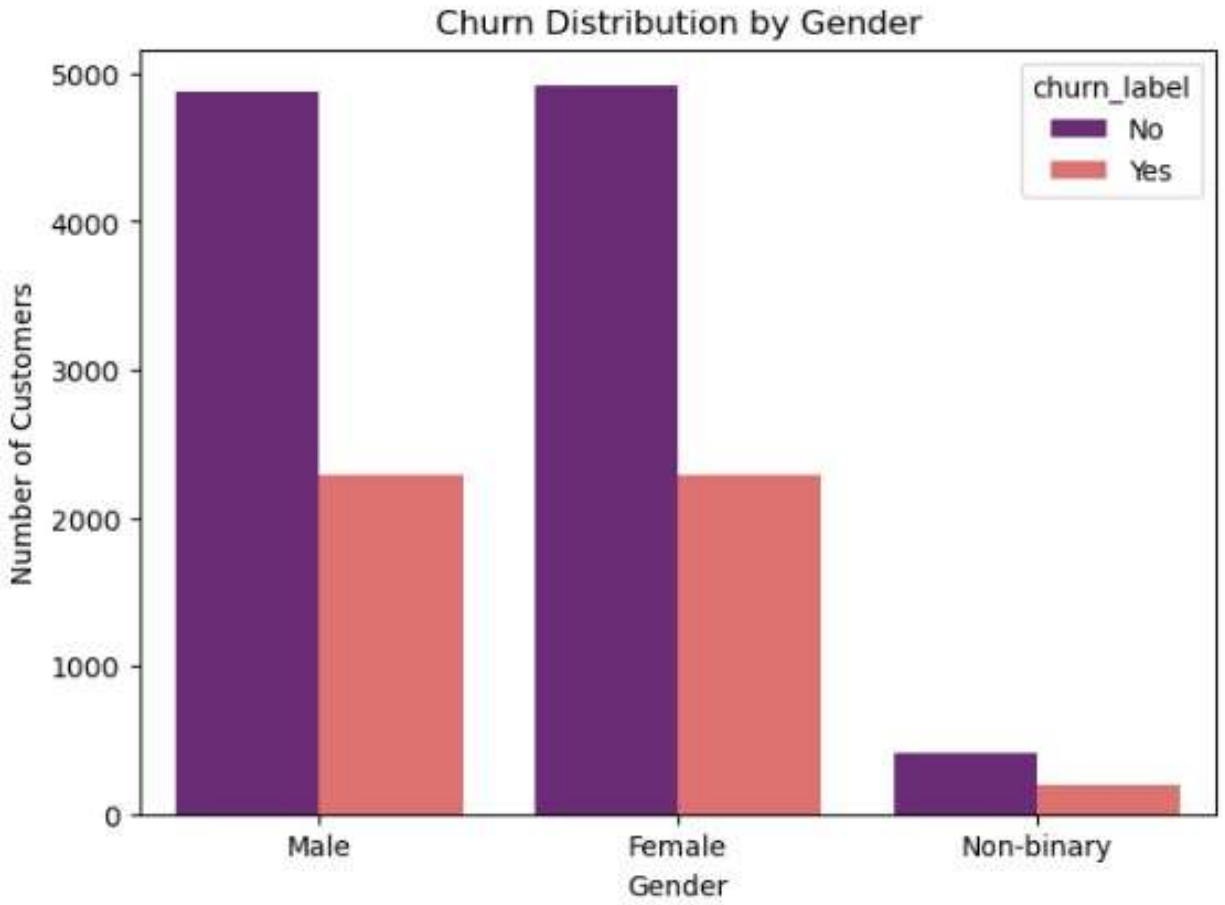
- Gender Distribution Analysis

```
#Shows the number of male and female customers

plt.figure(figsize=(7,5))

sns.countplot(x="gender", data = df, palette="magma")

plt.title("Customer Distribution by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Customers")

plt.show()
```



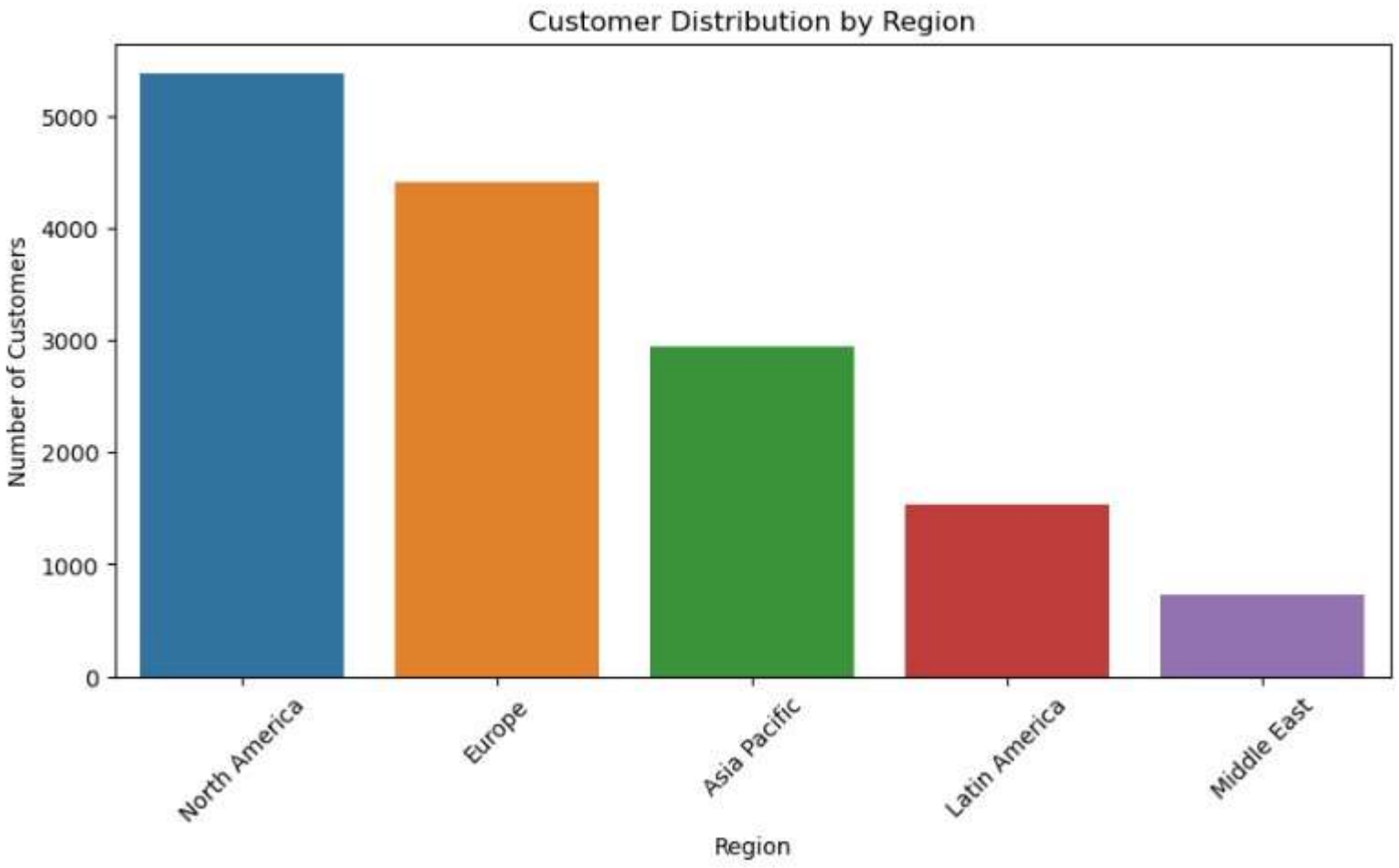
- Gender vs Churn Analysis

```
#Shows churn comparision between genders

df["churn_label"] = df["churn"].map({0: "No", 1: "Yes"})

plt.figure(figsize=(7,5))

sns.countplot(
    x="gender",
    hue="churn_label",
    data=df,
    palette="magma"
)

plt.title("Churn Distribution by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Customers")

plt.show()
```



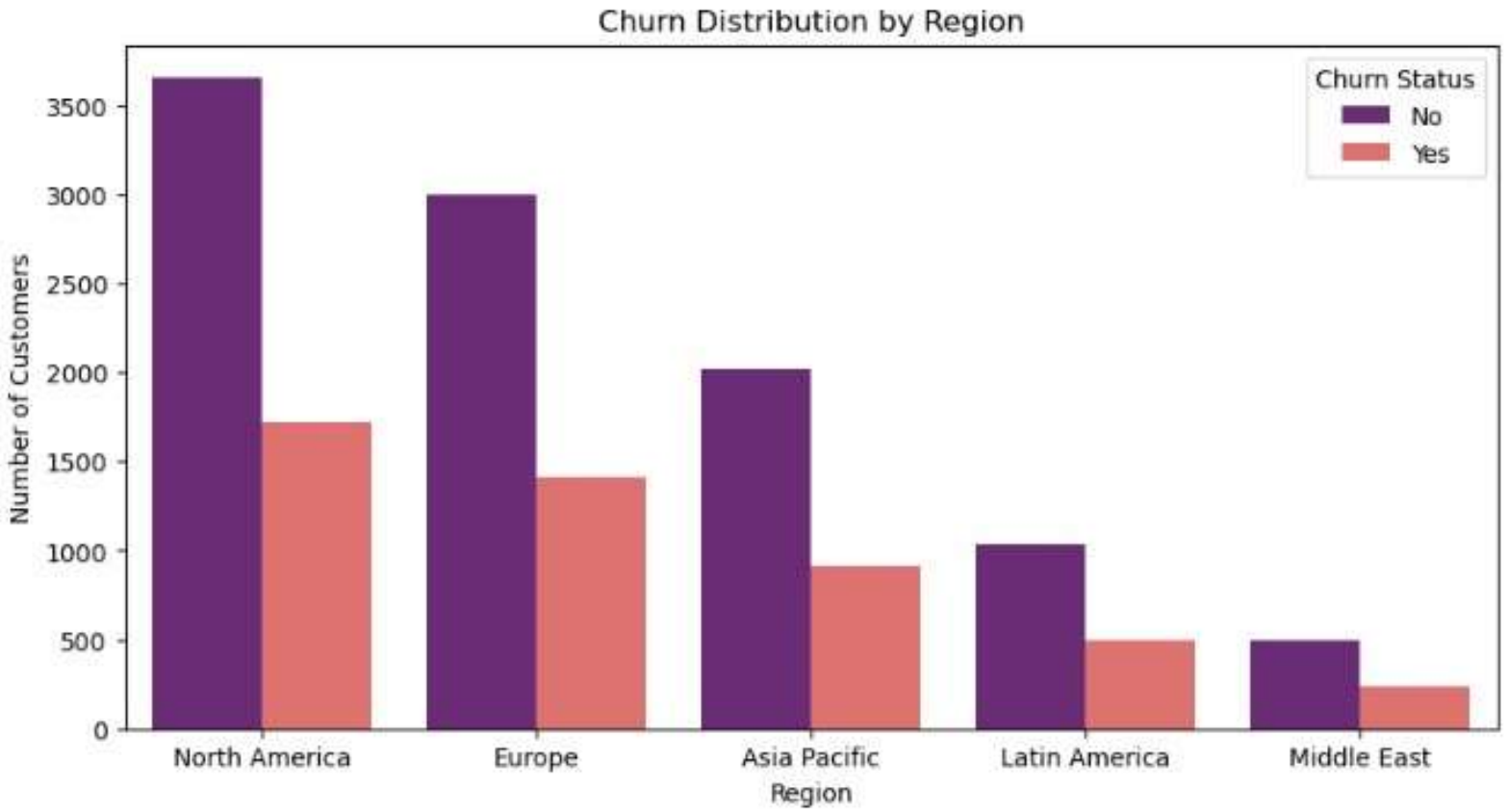
- Region distribution analysis

```
#shows customer distribution across regions

plt.figure(figsize=(10,5))

sns.countplot(x="region",data=df)

plt.title("Customer Distribution by Region")
plt.xlabel("Region")
plt.ylabel("Number of Customers")

plt.xticks(rotation=45)

plt.show()
```



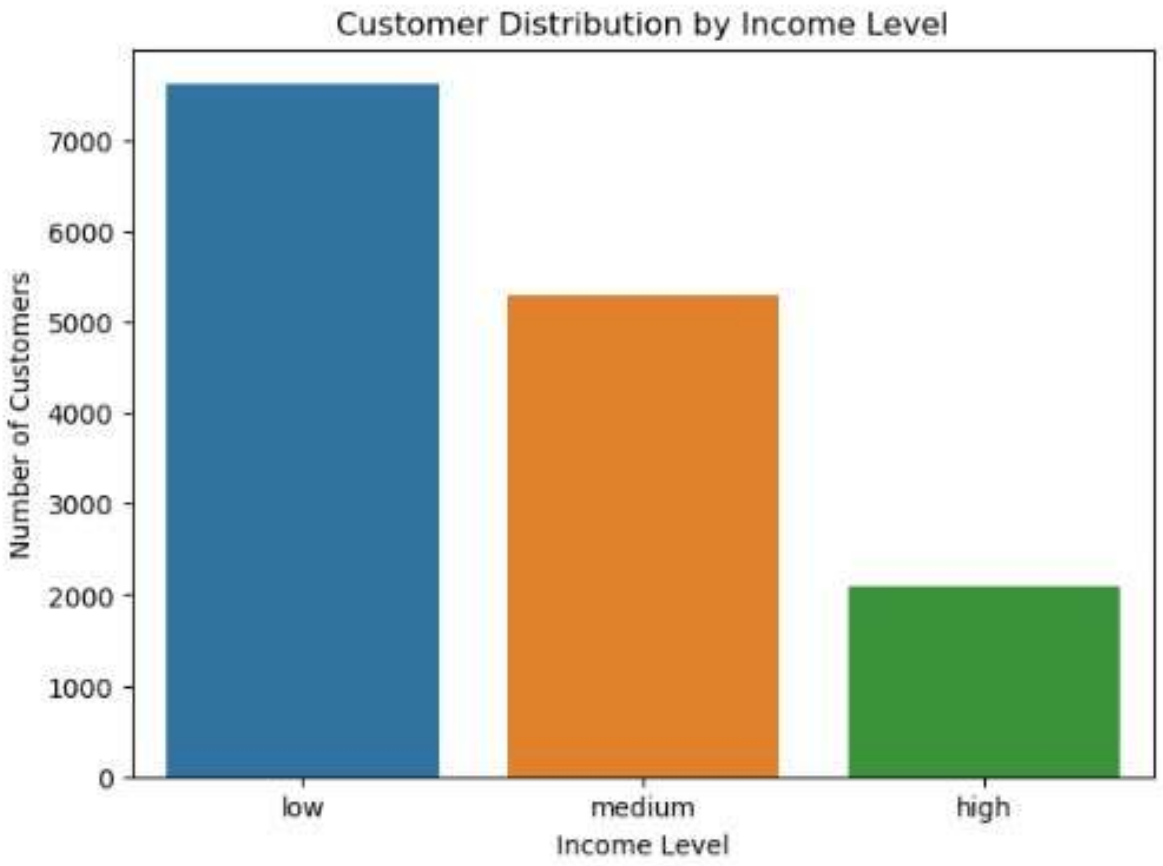
- Region vs Churn Analysis

```python
# This shows churn comparison across regions


# Create a separate string label for churn
df["churn_label"] = df["churn"].map({0: "No", 1: "Yes"})

# If churn contains other numeric codes, convert them to strings:
df["churn_label"] = df["churn_label"].astype(str)

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,5))

sns.countplot(
    x="region",
    hue="churn_label",   # use the new categorical column here
    data=df,
    palette="magma"
)

plt.title("Churn Distribution by Region")
plt.xlabel("Region")
plt.ylabel("Number of Customers")

plt.legend(title="Churn Status")   # optional, for a clearer legend
plt.show()
```



- Income level distribution

```
#shows how customers are distributed by income level

plt.figure(figsize=(7,5))

sns.countplot(x="income_level",data = df)

plt.title("Customer Distribution by Income Level")
plt.xlabel("Income Level")
plt.ylabel("Number of Customers")

plt.show()
```



- Summary Tables for Demographics

```python
#summary tables for demographics


#Age summary
print("\nAge Summary:")
print(df["age"].describe())

#Gender Count
print("\nGender Distribution:")
print(df["gender"].value_counts())

#Region Count
print("\nRegion Distribution:")
print(df["region"].value_counts())


#Income level Count
print("\nIncomeLevel Distribution:")
print(df["income_level"].value_counts())
```

```
Age Summary:
count    15000.000000
mean        41.646933
std         11.740994
min         18.000000
25%         33.000000
50%         42.000000
75%         50.000000
max         80.000000
Name: age, dtype: float64

Gender Distribution:
gender
Female        7210
Male          7171
Non-binary     619
Name: count, dtype: int64
```

```
Region Distribution:
region
North America    5377
Europe           4414
Asia Pacific     2938
Latin America    1537
Middle East       734
Name: count, dtype: int64

IncomeLevel Distribution:
income_level
low       7614
medium    5288
high      2098
Name: count, dtype: int64
```

# Customer Behavior Analysis

- Behavioral features in this dataset include:
- Usage_frequency → frequency of service usage
- Average_session_duration → average time spent per session
- Last_login_days → inactivity duration (higher values indicate greater inactivity)
- Support_tickets → number of customer complaints or support requests
- Satisfaction_score → customer satisfaction rating (1 to 5)

**Goal**:

Analyze customer engagement patterns and compare behavioral differences between retained and churned customers.

- Define Behavior Columns

```
#define behavior columns

behavior_columns=[
    "usage_frequency",
    "average_session_duration",
    "last_login_days",
    "support_tickets",
    "satisfaction_score"
]
```

- Define Behavior Columns & Show Statistical Summary

```python
#define correct behavior columns

behavior_columns=[
    "usage_frequency",
    "avg_session_duration_minutes",
    "number_of_support_tickets",
    "last_login_days_ago",
    "satisfaction_score"
]

#show statistical summary
print("Overall Behavior Summary")
print(df[behavior_columns].describe())
```

```
Overall Behavior Summary
       avg_session_duration_minutes  number_of_support_tickets  \
count                  15000.000000               15000.000000
mean                      45.235633                   2.927133
std                       29.244734                   2.609012
min                        5.000000                   0.000000
25%                       21.900000                   1.000000
50%                       40.500000                   2.000000
75%                       58.500000                   4.000000
max                      119.900000                  11.000000

       last_login_days_ago  satisfaction_score
count         15000.000000         15000.00000
mean             21.748533             3.38440
std              22.282570             1.24951
min               0.000000             1.00000
25%               5.000000             3.00000
50%              13.000000             4.00000
75%              32.000000             4.00000
max              89.000000             5.00000
```

- Compare Behavior By Churn

```python
#Compare Behavior by Churn

for col in behavior_columns:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# now compute average behavior by churn
behavior_churn_summary = df.groupby("churn")[behavior_columns].mean(numeric_only=True)

print("\nAverage Behavior by Churn:")
print(behavior_churn_summary)
```

```
Average Behavior by Churn:
       usage_frequency  avg_session_duration_minutes  \
churn
0                  NaN                     48.589508
1                  NaN                     38.071399

       number_of_support_tickets  last_login_days_ago  satisfaction_score
churn
0                       1.919154            12.546540            3.878144
1                       5.080284            41.404976            2.329709
```

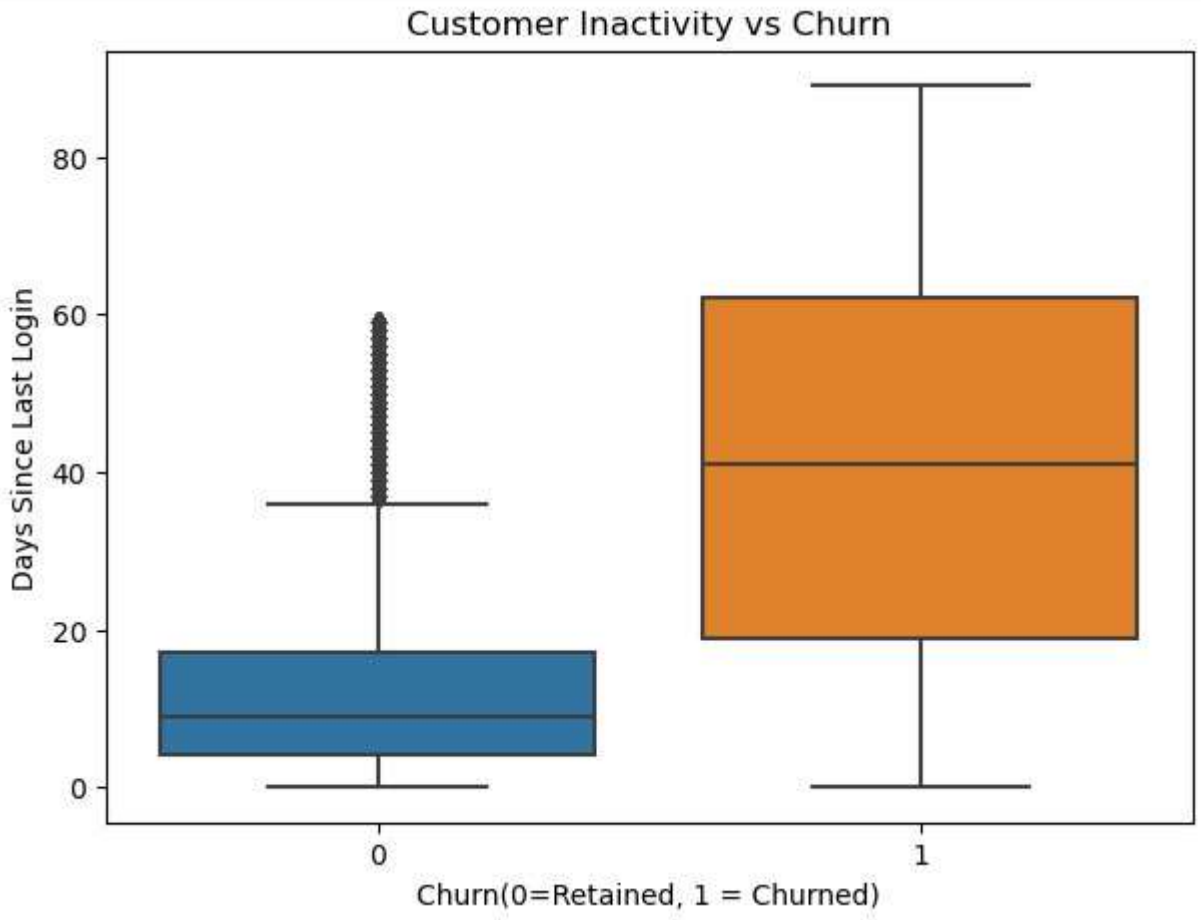
- Average Behavior of Retained vs Churned Customers

```
#shows average behavior of retained vs churned customers

behavior_churn_summary =  df.groupby("churn")[behavior_columns].mean()

print("\nAverage Behavior by Churn:")
print(behavior_churn_summary)
```

```
Average Behavior by Churn:
       usage_frequency  avg_session_duration_minutes  \
churn
0                  NaN                     48.589508
1                  NaN                     38.071399

       number_of_support_tickets  last_login_days_ago  satisfaction_score
churn
0                       1.919154            12.546540            3.878144
1                       5.080284            41.404976            2.329709
```

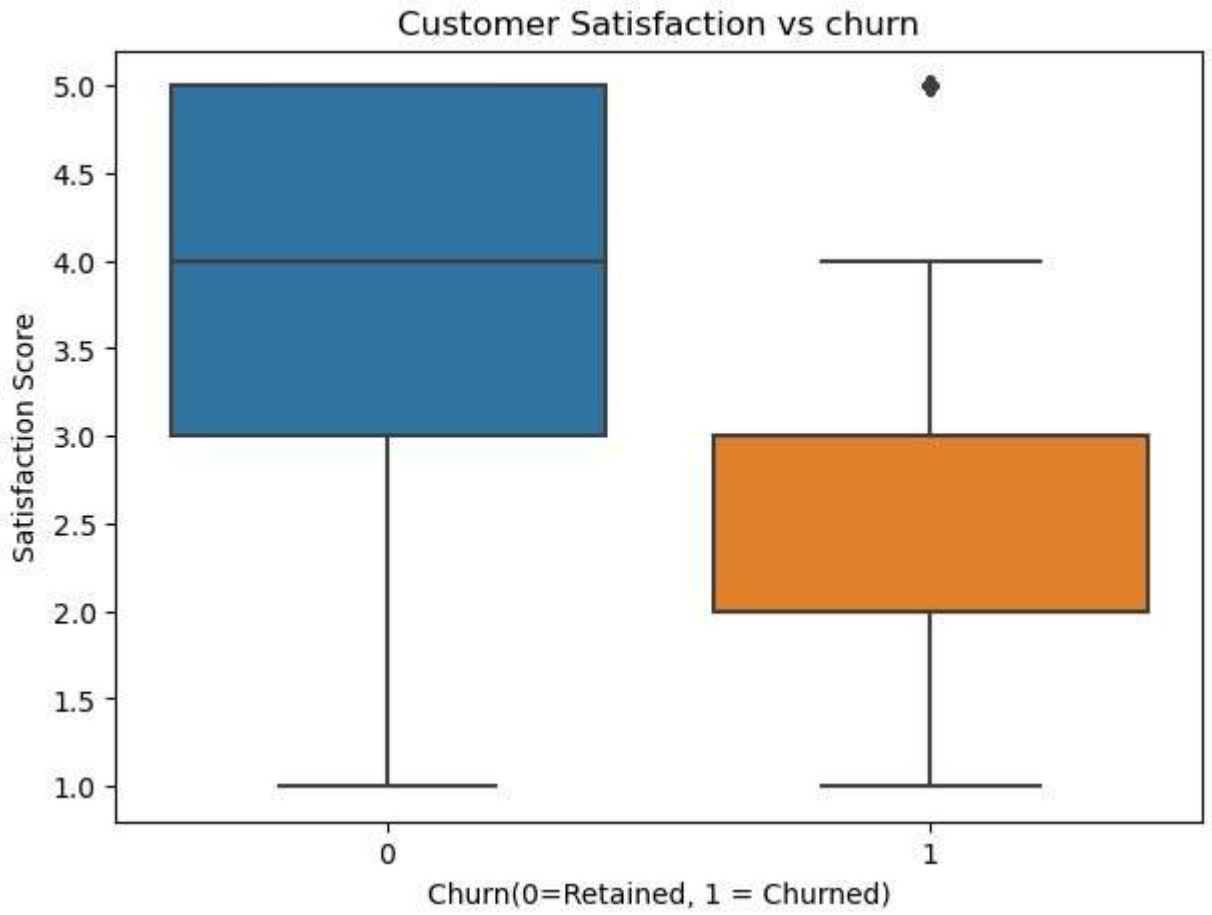
- Customer Inactivity vs Churn

```
#Customer Inactivity vs Churn

#Clean data before plotting

df_plot = df.replace([np.inf,-np.inf],np.nan)

df_plot = df_plot.dropna(subset=["churn","last_login_days_ago"])

#create boxplot

plt.figure(figsize=(7,5))

sns.boxplot(x="churn",y="last_login_days_ago",data=df_plot)

plt.title("Customer Inactivity vs Churn")
plt.xlabel("Churn(0=Retained, 1 = Churned)")
plt.ylabel("Days Since Last Login")

plt.show()
```



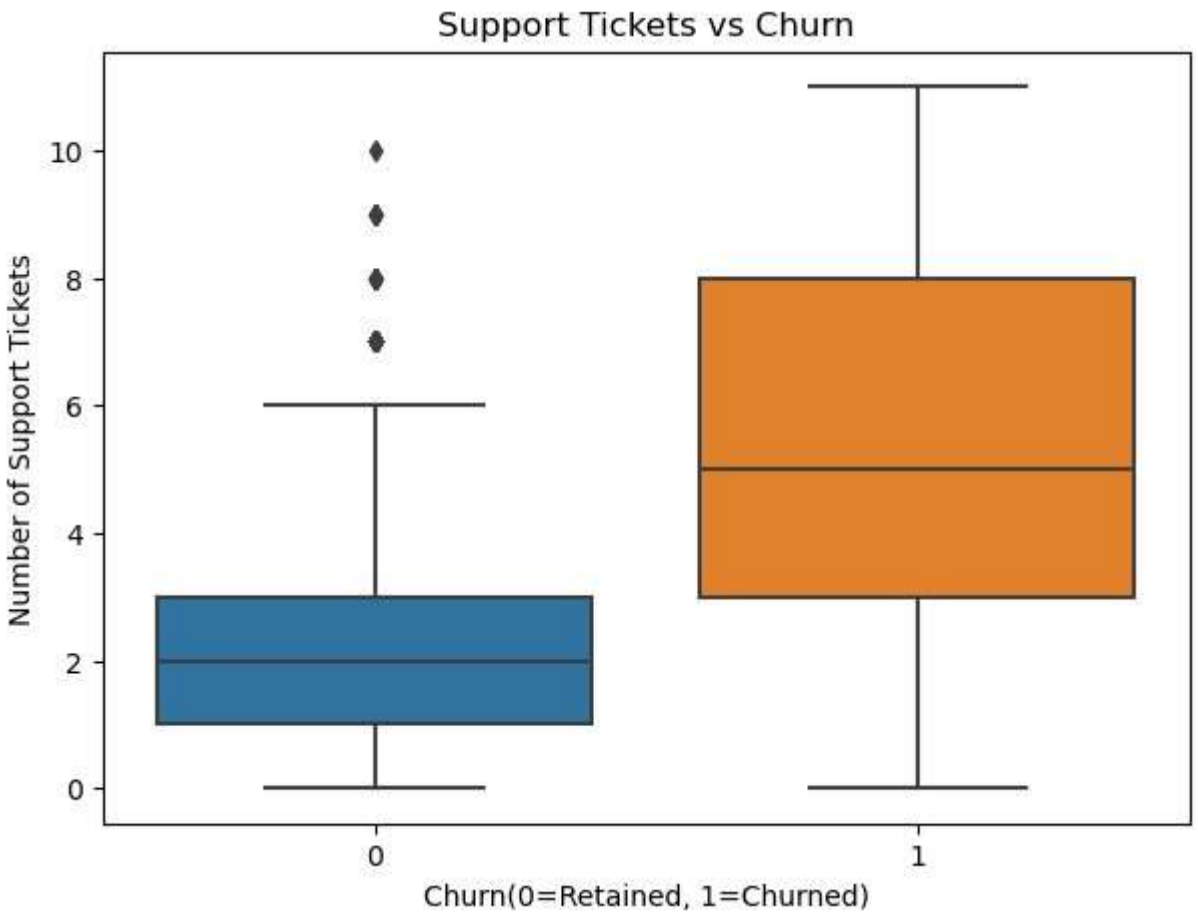
- Satisfaction Score vs Churn

```
#satisfaction score vs churn

#shows wheather dissatisfied customers churn more

plt.figure(figsize=(7,5))
sns.boxplot(x="churn",y="satisfaction_score",data = df)

plt.title("Customer Satisfaction vs churn")
plt.xlabel("Churn(0=Retained, 1 = Churned)")
plt.ylabel("Satisfaction Score")

plt.show()
```



- Support Tickets vs Churn

```
#support tickets vs Churn (Correct)

#clean data before plotting

df_plot=df.replace([np.inf,-np.inf],np.nan)
df_plot=df_plot.dropna(subset=["churn","number_of_support_tickets"])

#Create boxplot

plt.figure(figsize=(7,5))

sns.boxplot(x="churn",y="number_of_support_tickets",data=df_plot)


plt.title("Support Tickets vs Churn")
plt.xlabel("Churn(0=Retained, 1=Churned)")
plt.ylabel("Number of Support Tickets")

plt.show()
```



# Contract, Billing & Service Insights

This section analyzes **subscription and billing features**:
**subscription_type** → basic / standard / premium
**payment_method** → payment type used by the customer
**monthly_charges** → monthly cost of the subscription
**total_spent** → total amount spent by the customer
**discount_usage** → whether the customer used a discount
**promotional_response** → whether the customer responded to a promotion

**Goal:**
Understand how subscription plans, billing behavior, and promotional offers are related to customer churn.

- Define Billing and Service Columns & Show Number of Customers in Each Subscription Plan

```python
#define billing and service columns
billing_columns=[
    "subscription_type",
    "payment_method",
    "monthly_charges",
    "total_spent",
    "discount_usage",
    "promotional_response"

]
```

```python
#this shows number of customers in each subscription plan

print("subscription Type Distribution:")
print(df["subscription_type"].value_counts())
```

```
subscription Type Distribution:
subscription_type
basic       6428
standard    6107
premium     2465
Name: count, dtype: int64
```

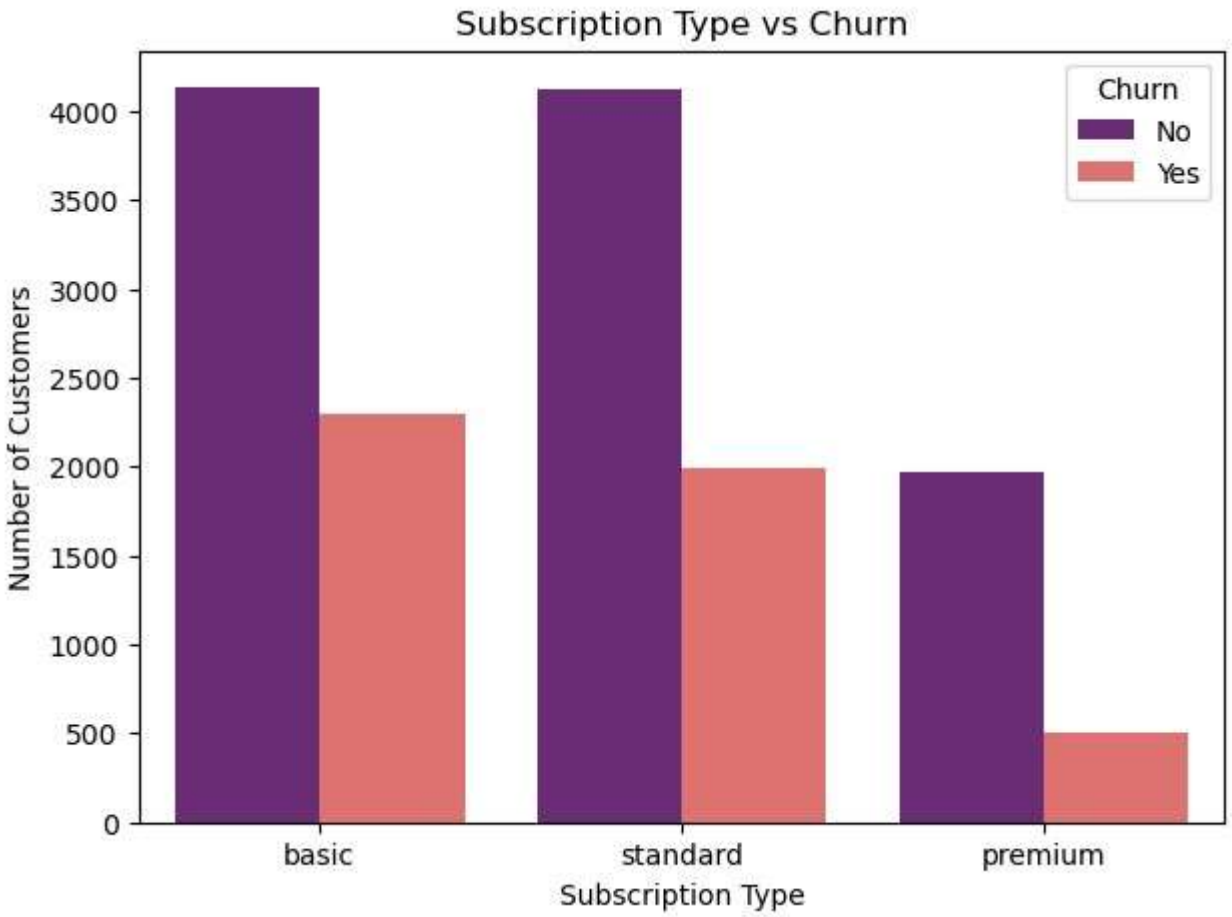
- Churn Rate By Subscription Type

```
#This shows churn rate by subscription type

subscription_churn = pd.crosstab(df["subscription_type"],df["churn"])

print("\nSubscription Type vs Churn:")
print(subscription_churn)
```

```
Subscription Type vs Churn:
churn                 0     1
subscription_type
basic              4134  2294
premium            1965   500
standard           4118  1989
```

- Subscription Type vs Churn

```python
#visualization


# Make sure churn is categorical text
df["churn_label"] = df["churn"].map({0: "No", 1: "Yes"}).astype(str)

plt.figure(figsize=(7,5))

sns.countplot(
    x="subscription_type",
    hue="churn_label",   # <-- use the string column here
    data=df,
    palette="magma"
)

plt.title("Subscription Type vs Churn")
plt.xlabel("Subscription Type")
plt.ylabel("Number of Customers")

plt.legend(title="Churn")
plt.show()
```

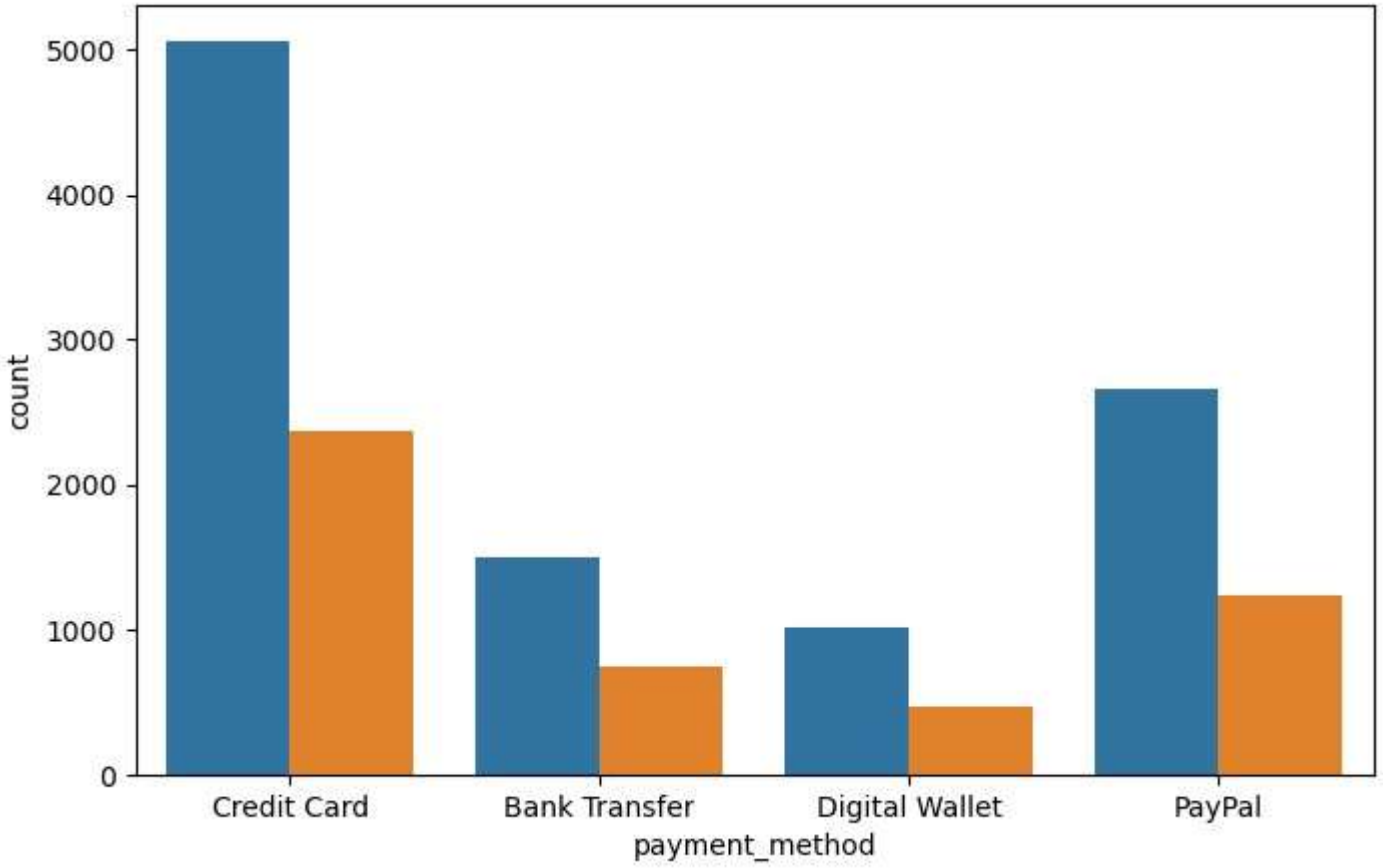


- Churn Pattern By Payment Method

```
#shows churn pattern by payment method

payment_churn=pd.crosstab(df["payment_method"],df["churn"])

print("\nPayement Method vs Churn:")
print(payment_churn)
```

```
Payement Method vs Churn:
churn              0     1
payment_method
Bank Transfer   1493   734
Credit Card     5057  2360
Digital Wallet  1011   457
PayPal          2656  1232
```

```
#visualization

plt.figure(figsize=(8,5))

sns.countplot(x="payment_method",hue="churn",data =df)

plt.title("Payment Method vs Churn")
plt.xlabel("Payment Method")
plt.ylabel("Number of Customers")

plt.xticks(rotation=45)
plt.show()
```

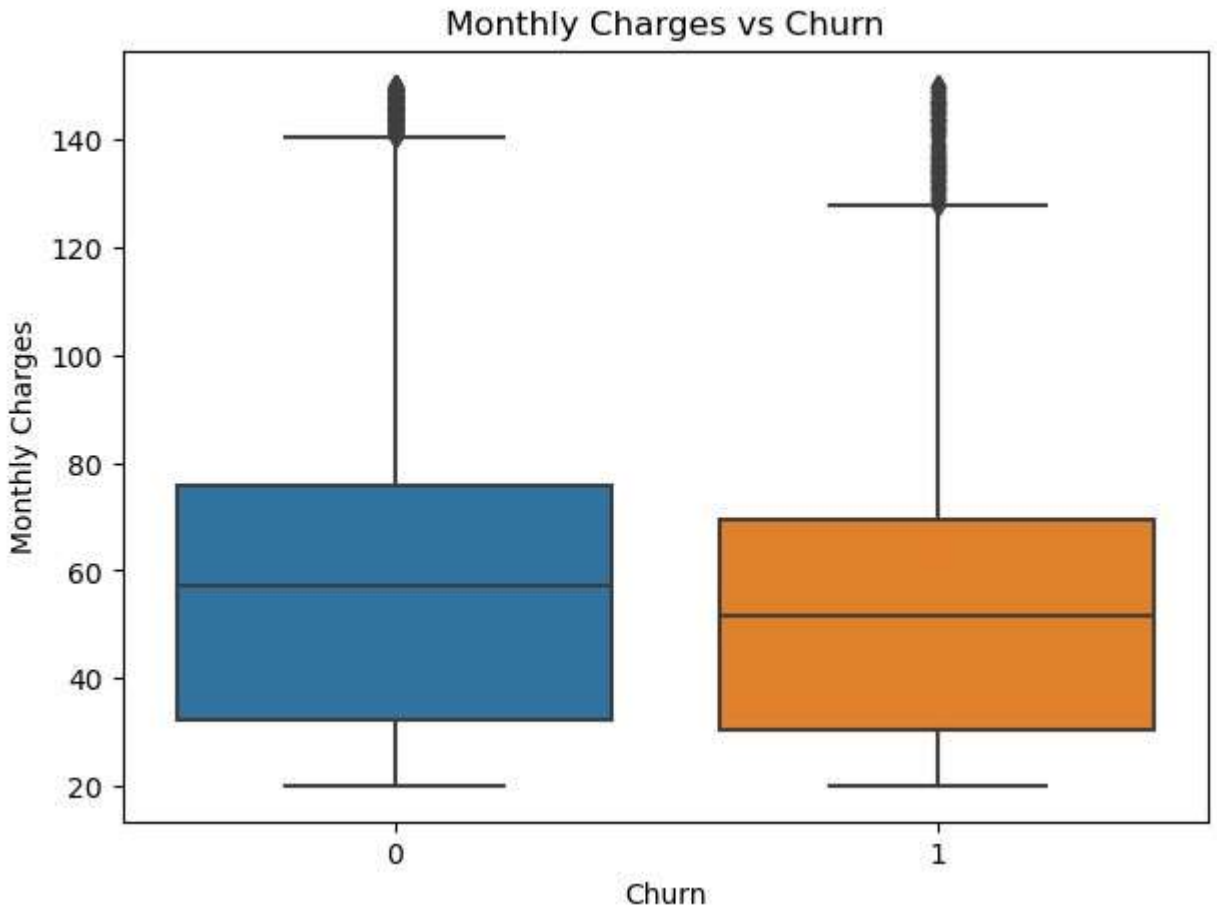


- Monthly Summary By Churn

```
#shows monthly charges summary by churn

print("\nMonthly Charge Summary by Churn:")
print(df.groupby("churn")["monthly_charges"].describe())
```

```
Monthly Charge Summary by Churn:
         count       mean        std    min    25%    50%     75%     max
churn
0      10217.0  61.469184  34.257263  19.99  32.46  57.14  75.670  149.95
1       4783.0  53.870366  29.470553  20.00  30.36  51.57  69.345  149.89
```

```
#visualization

plt.figure(figsize=(7,5))

sns.boxplot(x="churn",y="monthly_charges",data=df)

plt.title("Monthly Charges vs Churn")
plt.xlabel("Churn")
plt.ylabel("Monthly Charges")

plt.show()
```



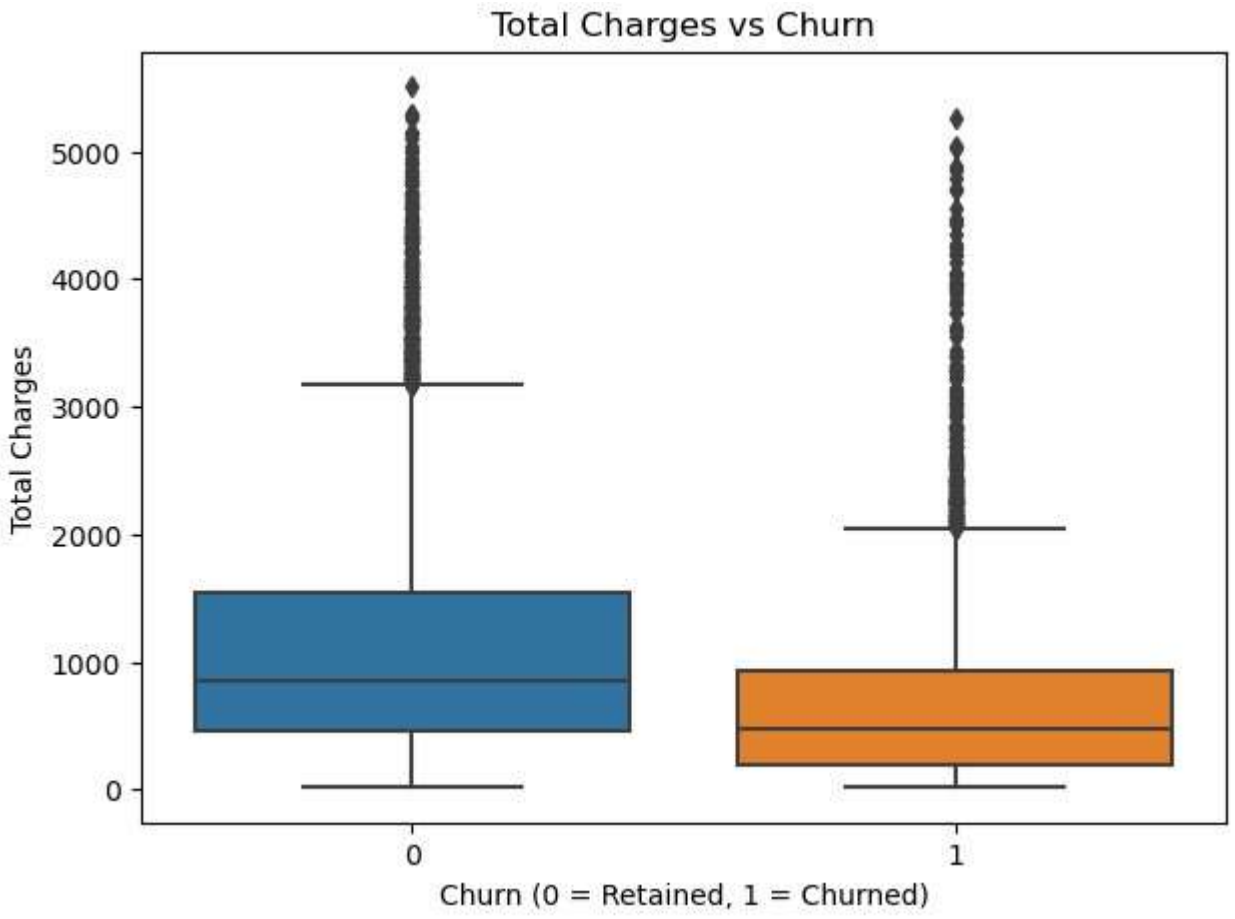
- Total Charges Summary By Churn

```
#Total charges summary by churn(correct)

#Convert tonumeric(safe practice)

df["total_charges"]=pd.to_numeric(df["total_charges"],errors="coerce")


#show summary statistics grouped by churn

print("\nTotal Charges Summary by Churn:")

print(df.groupby("churn")["total_charges"].describe())
```

```
Total Charges Summary by Churn:
         count         mean         std    min      25%     50%      75%  \
churn
0      10217.0  1112.202036  902.038958  16.95  466.110  855.76  1546.57
1       4783.0   691.984052  696.712290  16.50  194.245  474.80   934.53

           max
churn
0      5506.35
1      5255.64
```

- Total Charges vs Churn

```
#clean data before plotting

df_plot=df.replace([np.inf,-np.inf],np.nan)

df_plot=df_plot.dropna(subset=["churn","total_charges"])


#create boxplot
plt.figure(figsize=(7,5))

sns.boxplot(x="churn", y="total_charges", data=df_plot)

plt.title("Total Charges vs Churn")
plt.xlabel("Churn (0 = Retained, 1 = Churned)")
plt.ylabel("Total Charges")

plt.show()
```



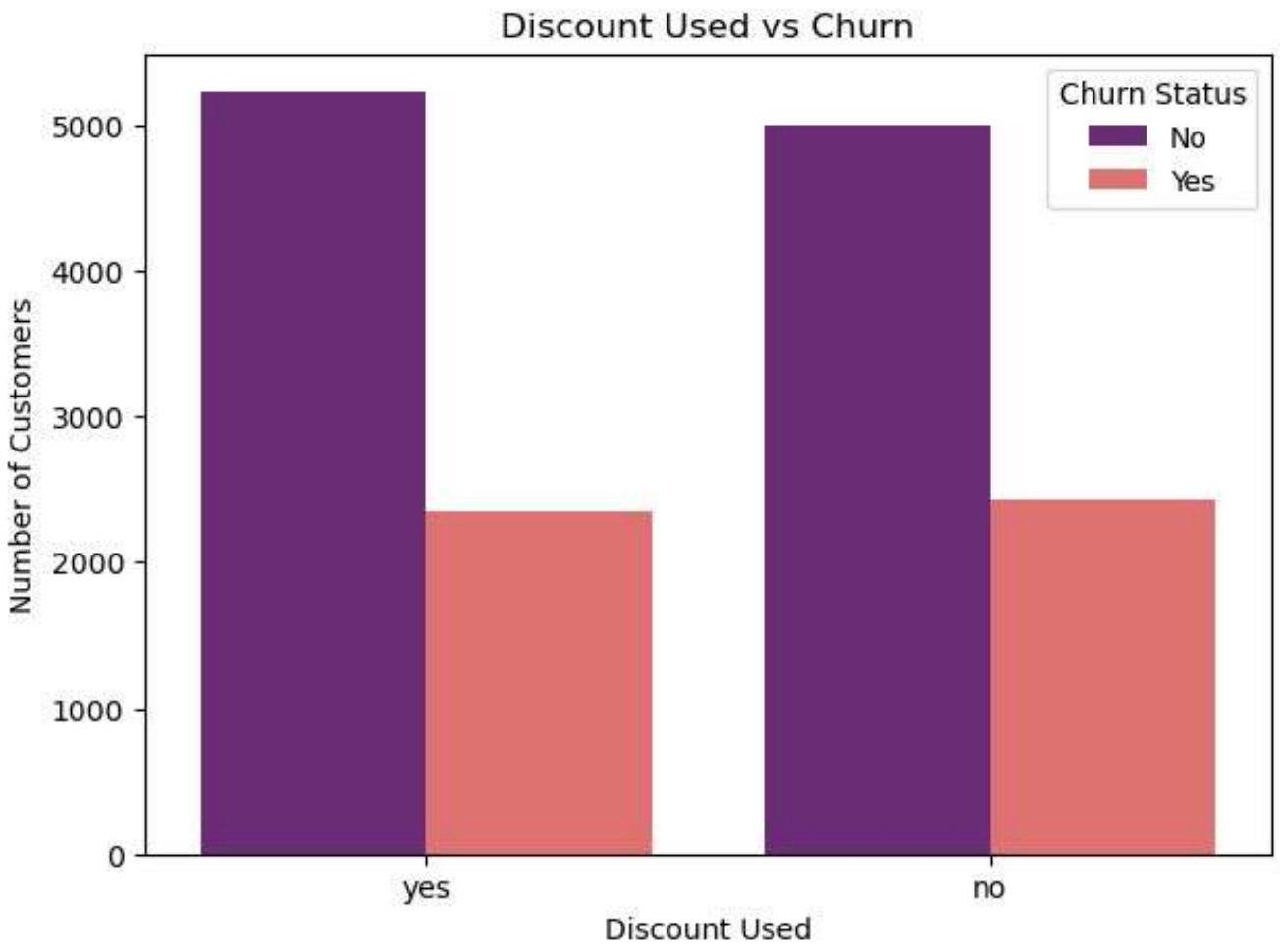
- Discount Used vs Churn

```
#discount Used vs Churn (Correct)

#Create churn comparision table

discount_churn = pd.crosstab(df["discount_used"],df["churn"])

print("\nDiscount Used vs Churn:")
print(discount_churn)
```

```
Discount Used vs Churn:
churn              0     1
discount_used
no              4993  2437
yes             5224  2346
```

- Discount Used vs Churn (Visualization)

```
#clean data before plotting

df_plot=df.replace([np.inf,-np.inf],np.nan)

df_plot=df_plot.dropna(subset=["discount_used","churn"])

#create countplot

plt.figure(figsize=(7,5))
sns.countplot(
    x="discount_used",
    hue="churn_label",   # use string labels
    data=df,
    palette="magma"
)

plt.title("Discount Used vs Churn")
plt.xlabel("Discount Used")
plt.ylabel("Number of Customers")
plt.legend(title="Churn Status")
plt.show()
```

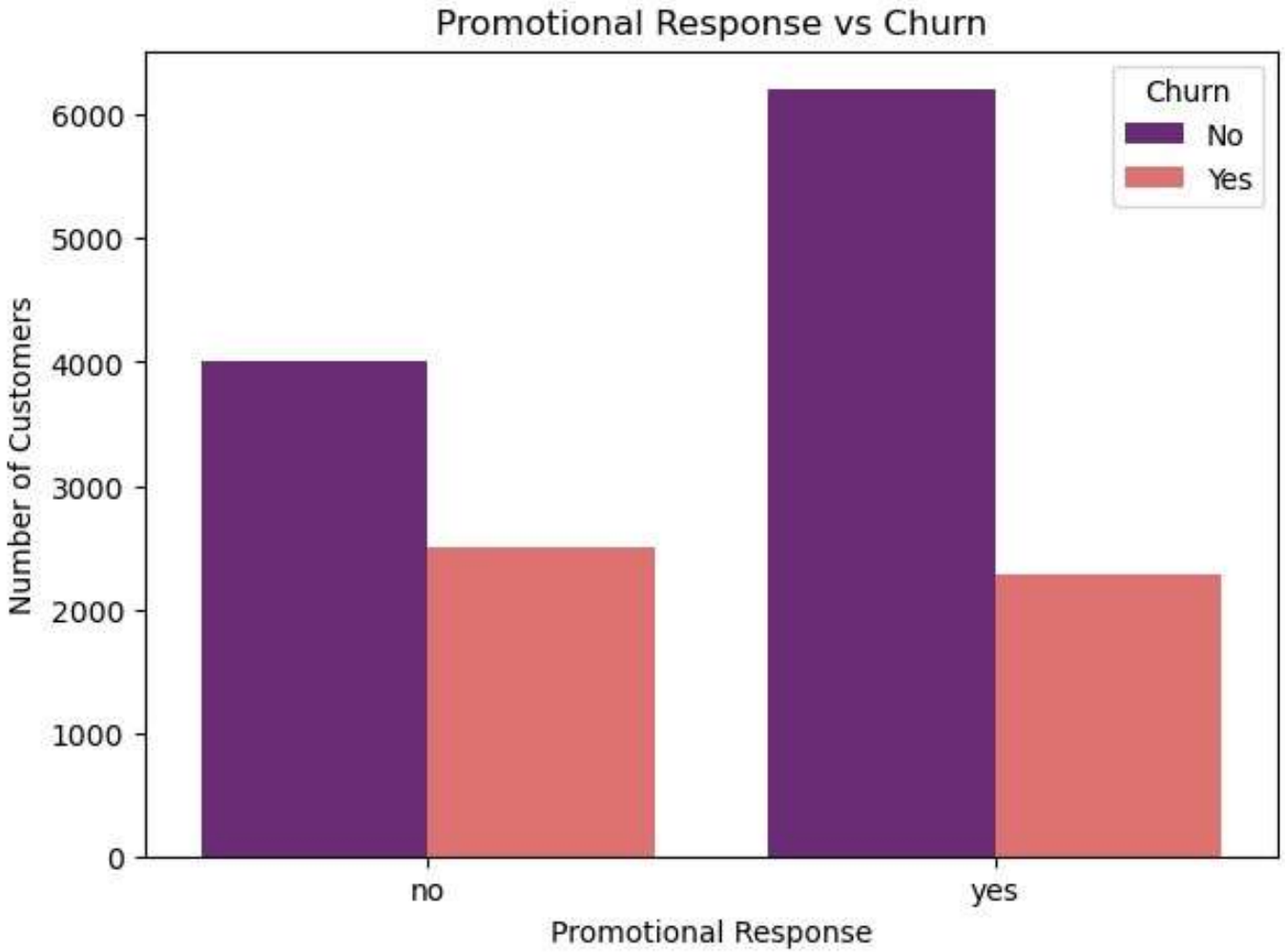


- Promotions Improve Retention

```
#shows wheather promotions improve retention

promo_churn=pd.crosstab(df["promotional_response"],df["churn"])

print("\nPromotional Response vs Churn:")
print(promo_churn)
```

```
Promotional Response vs Churn:
churn                    0     1
promotional_response
no                    4010  2503
yes                   6207  2280
```

```
#visualization

plt.figure(figsize=(7,5))

sns.countplot(
    x="promotional_response",
    hue="churn_label",  # use the string column
    data=df,
    palette="magma"
)

plt.title("Promotional Response vs Churn")
plt.xlabel("Promotional Response")
plt.ylabel("Number of Customers")
plt.legend(title="Churn")

plt.show()
```



# UNIVARIATE DISTRIBUTION ANALYSIS

- Univariate analysis examines the distribution of individual variables.
- We analyze each feature separately to understand its distribution pattern, central tendency, and spread.

- Based on this dataset, we will analyze important numerical features:

- age→ age of the customer
- monthly_charges → monthly subscription cost
- total_spent → total amount spent by the customer
- usage_frequency → how frequently the service is used
- average_session_duration → average duration of each session
- last_login_days → number of days since the last login
- support_tickets → number of support requests raised
- satisfaction_score → customer satisfaction rating

- Making Numerical Columns List

```
#Correct NumericalColumns List

numerical_columns =[
    "tenure_months",
    "age",
    "monthly_charges",
    "total_charges",
    "usage_frequency",
    "avg_session_duration_minutes",
    "number_of_logins_per_month",
    "number_of_support_tickets",
    "satisfaction_score",
    "last_login_days_ago",
    "tenure_months"
]
```

- Statistical Summary of Numerical Features

```
#shows count,mean,std,min,max and quartiles
print("Statistical Summary of Numerical Features:")
print(df[numerical_columns].describe())
```

Statistical Summary of Numerical Features:

| | tenure_months | age | monthly_charges | total_charges |
|---|---|---|---|---|
| count | 15000.000000 | 15000.000000 | 15000.000000 | 15000.000000 |
| mean | 17.448467 | 41.646933 | 59.046174 | 978.208528 |
| std | 10.304214 | 11.740994 | 32.996490 | 864.474724 |
| min | 1.000000 | 18.000000 | 19.990000 | 16.500000 |
| 25% | 8.000000 | 33.000000 | 31.640000 | 351.145000 |
| 50% | 17.000000 | 42.000000 | 55.150000 | 735.260000 |
| 75% | 26.000000 | 50.000000 | 73.742500 | 1372.332500 |
| max | 35.000000 | 80.000000 | 149.950000 | 5506.350000 |

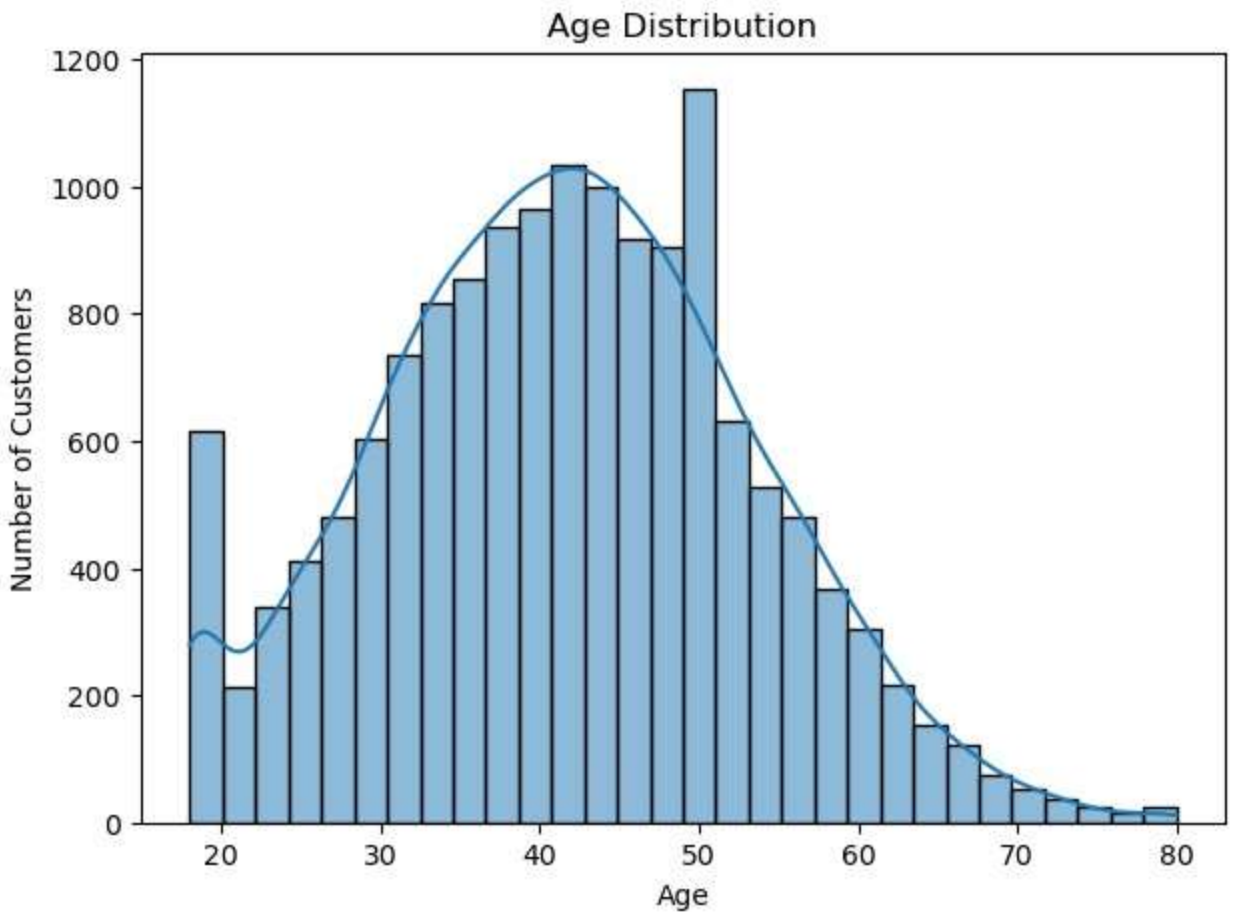
| | satisfaction_score | last_login_days_ago | tenure_months |
|---|---|---|---|
| count | 15000.00000 | 15000.000000 | 15000.000000 |
| mean | 3.38440 | 21.748533 | 17.448467 |
| std | 1.24951 | 22.282570 | 10.304214 |
| min | 1.00000 | 0.000000 | 1.000000 |
| 25% | 3.00000 | 5.000000 | 8.000000 |
| 50% | 4.00000 | 13.000000 | 17.000000 |
| 75% | 4.00000 | 32.000000 | 26.000000 |
| max | 5.00000 | 89.000000 | 35.000000 |

| | usage_frequency | avg_session_duration_minutes |
|---|---|---|
| count | 0.0 | 15000.000000 |
| mean | NaN | 45.235633 |
| std | NaN | 29.244734 |
| min | NaN | 5.000000 |
| 25% | NaN | 21.900000 |
| 50% | NaN | 40.500000 |
| 75% | NaN | 58.500000 |
| max | NaN | 119.900000 |

| | number_of_logins_per_month | number_of_support_tickets |
|---|---|---|
| count | 15000.000000 | 15000.000000 |
| mean | 18.519600 | 2.927133 |
| std | 12.335911 | 2.609012 |
| min | 1.000000 | 0.000000 |
| 25% | 9.000000 | 1.000000 |
| 50% | 16.000000 | 2.000000 |
| 75% | 24.000000 | 4.000000 |
| max | 49.000000 | 11.000000 |

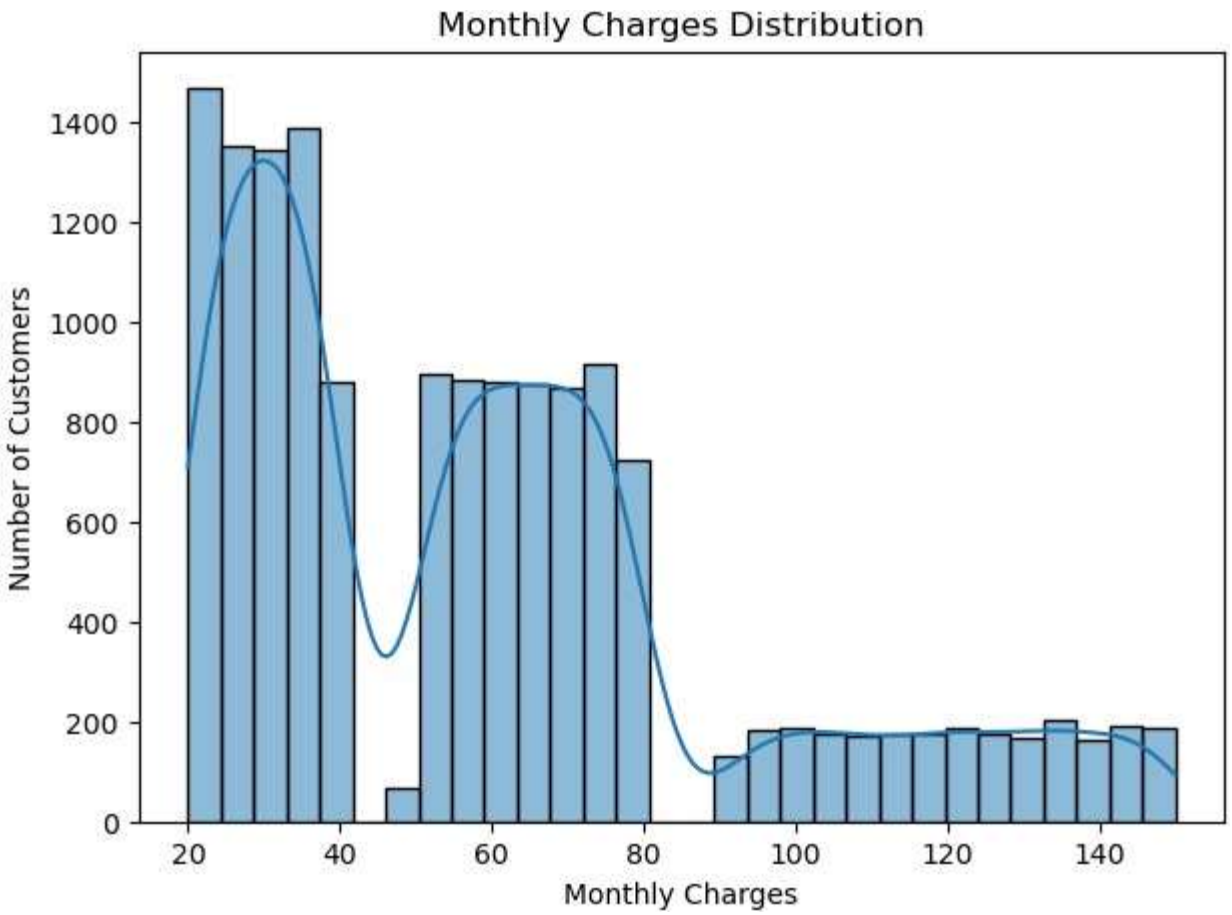
- Age Distribution

```
#Age Distribution

plt.figure(figsize=(7,5))

sns.histplot(df["age"],bins=30,kde=True)

plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Number of Customers")

plt.show()
```



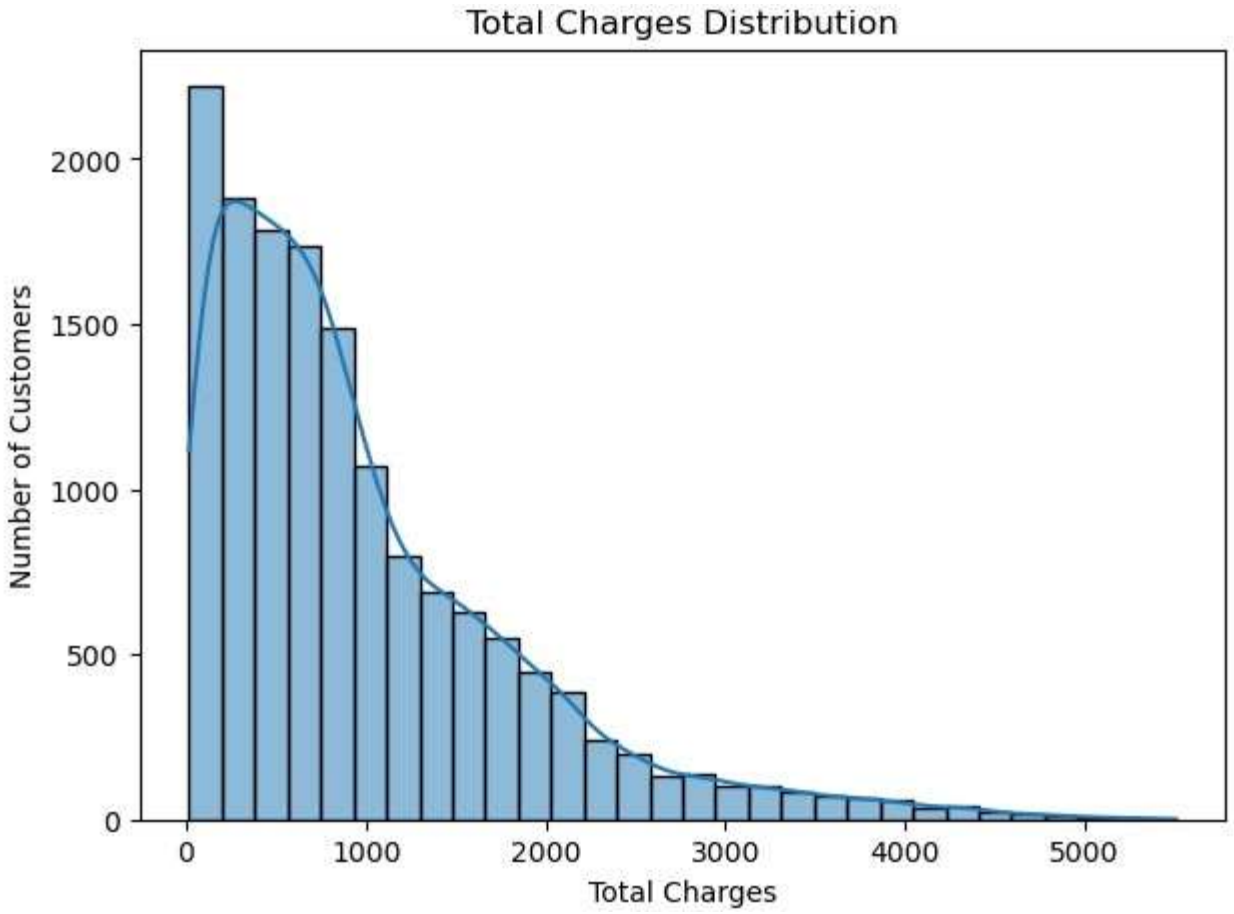
- Monthly Charges Distribution

```
#Monthly Charges Distribution

plt.figure(figsize=(7,5))

sns.histplot(df["monthly_charges"],bins=30,kde=True)

plt.title("Monthly Charges Distribution")
plt.xlabel("Monthly Charges")
plt.ylabel("Number of Customers")

plt.show()
```



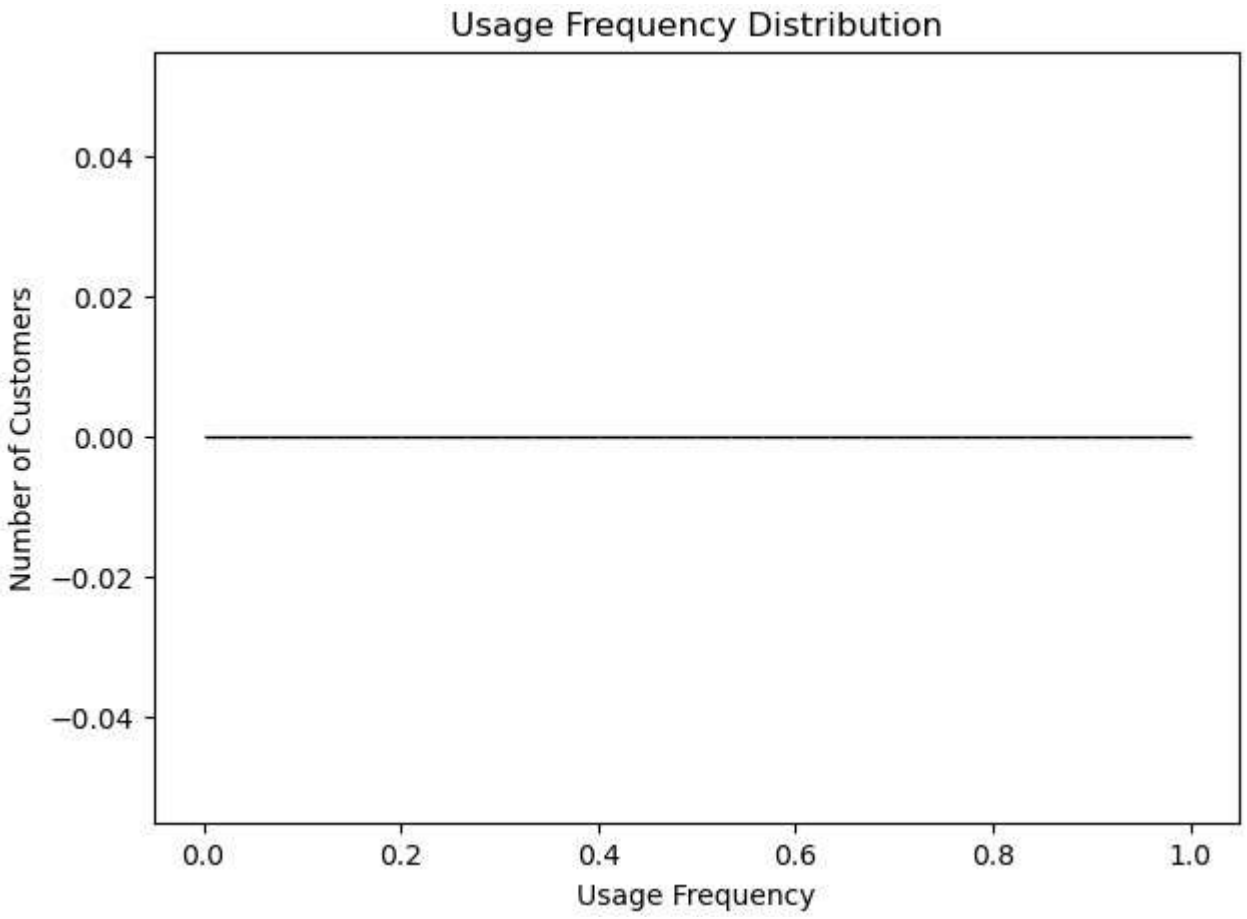
- Total Charges Distribution

```
#Total Spent Distribution

#Total Charges Distribution (Correct)

#Convert tonumeric (safe practice)

df["total_charges"]= pd.to_numeric(df["total_charges"], errors ="coerce")

#clean missing values

df_plot = df.dropna(subset =["total_charges"])

#Create histogram

plt.figure(figsize=(7,5))

sns.histplot(df_plot["total_charges"],bins=30,kde=True)

plt.title("Total Charges Distribution")
plt.xlabel("Total Charges")
plt.ylabel("Number of Customers")

plt.show()
```



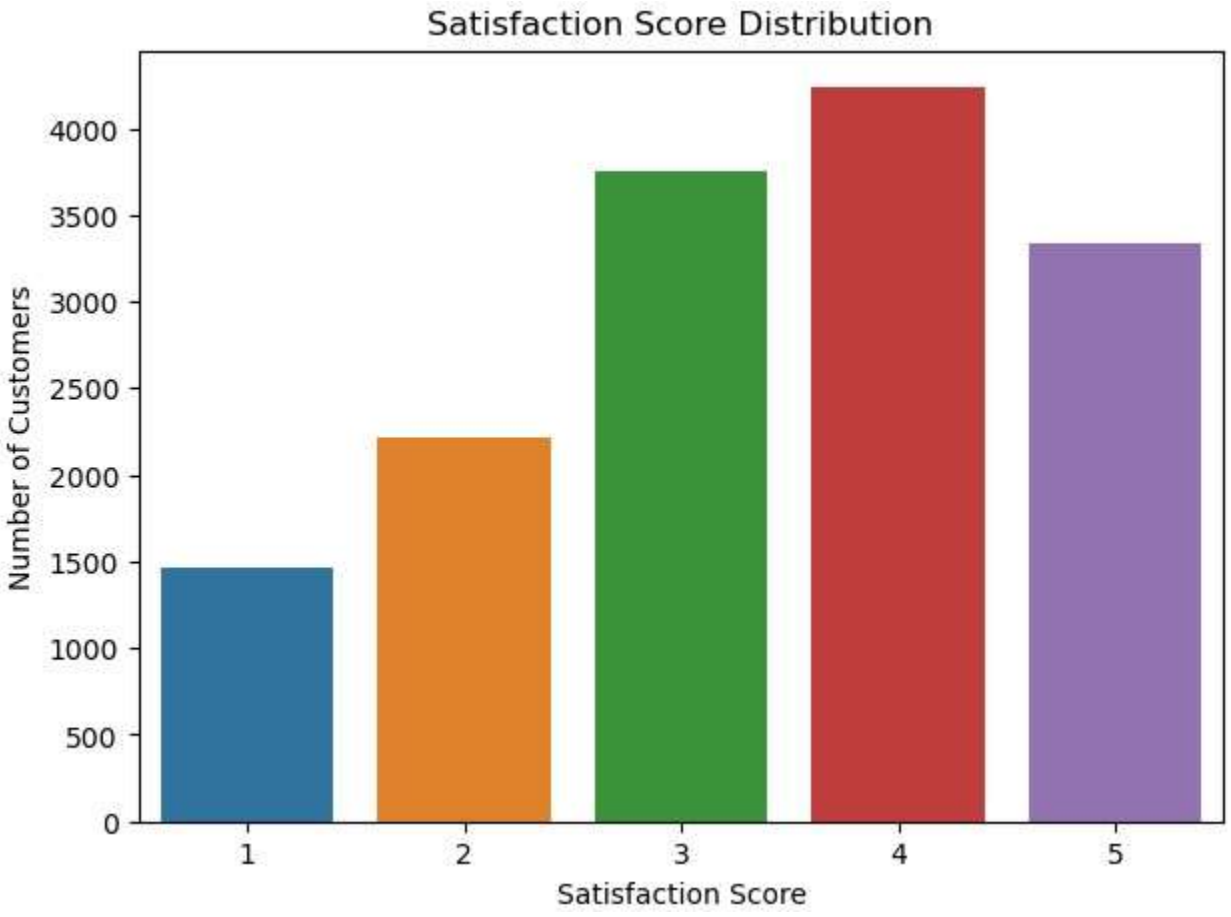
- Usage Frequency Distribution

```
#Usage Frequency Distribution

plt.figure(figsize=(7,5))

sns.histplot(df["usage_frequency"],bins=30,kde=True)

plt.title("Usage Frequency Distribution")
plt.xlabel("Usage Frequency")
plt.ylabel("Number of Customers")

plt.show()
```



- Satisfaction Score Distribution

```
#Satisfaction Score Distribution

#Satisfaction score is categoricalnumeric (1 to 5)

plt.figure(figsize=(7,5))

sns.countplot(x="satisfaction_score",data=df)

plt.title("Satisfaction Score Distribution")
plt.xlabel("Satisfaction Score")
plt.ylabel("Number of Customers")

plt.show()
```



# Feature-wise Churn Impact Analysis

- Identify which features have strongest impact on churn.
- We use summary tables for all features and graphs only for key churn drivers.

- Making Numerical Feature List

```
#define numerical  features(Correct for your dataset)

numerical_features = [
    "tenure_months",
    "age",
    "monthly_charges",
    "total_charges",
    "usage_frequency",
    "avg_session_duration_minutes",
    "number_of_logins_per_month",
    "number_of_support_tickets",
    "satisfaction_score",
    "last_login_days_ago",
    "tenure_months"
]
```

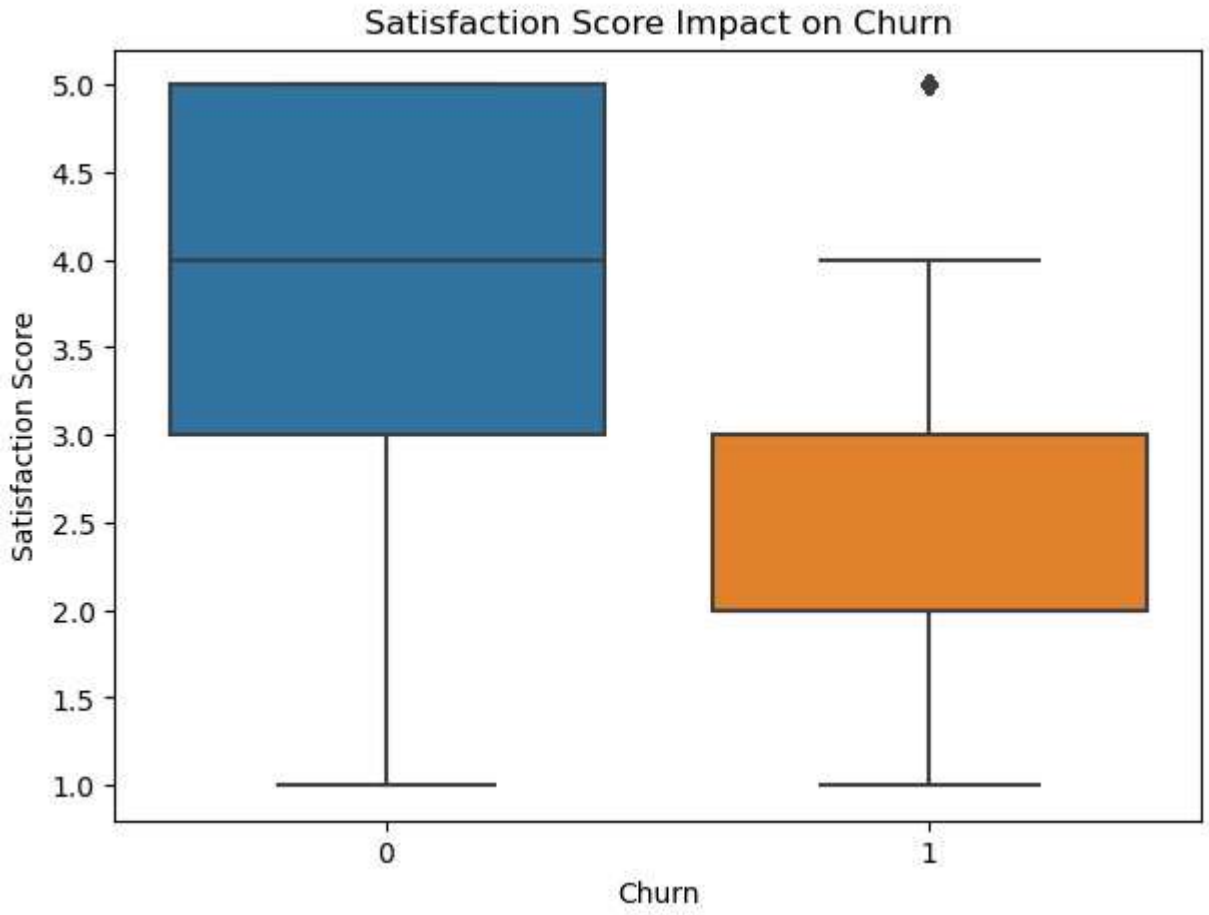
- Numerical Feature Impact

```python
#Numerical featureimpact on churn

#Compare average values between retained and churned customers

numerical_impact = df.groupby("churn")[numerical_features].mean()

print("Numerical Feature Impact on Churn:")
print(numerical_impact)
```

```
Numerical Feature Impact on Churn:
       tenure_months        age  monthly_charges  total_charges  \
churn
0          19.133601  41.591563        61.469184    1112.202036
1          13.848840  41.765210        53.870366     691.984052

       usage_frequency  avg_session_duration_minutes  \
churn
0                  NaN                     48.589508
1                  NaN                     38.071399

       number_of_logins_per_month  number_of_support_tickets  \
churn
0                       19.871391                   1.919154
1                       15.632030                   5.080284

       satisfaction_score  last_login_days_ago  tenure_months
churn
0                3.878144            12.546540      19.133601
1                2.329709            41.404976      13.848840
```

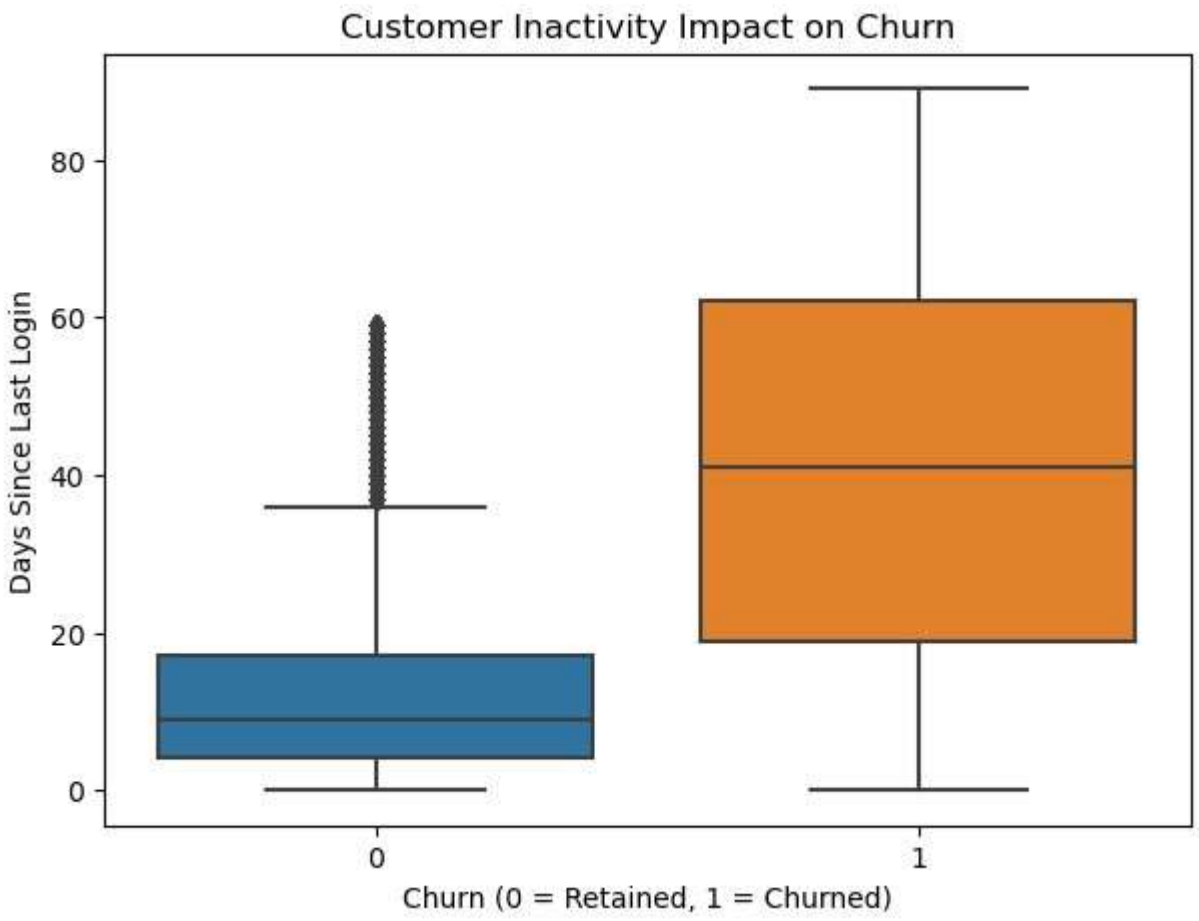
- Satisfaction Score Impact

```
#satisfaction Score Impact (IMP)

plt.figure(figsize=(7,5))

sns.boxplot(x="churn",y="satisfaction_score",data=df)

plt.title("Satisfaction Score Impact on Churn")
plt.xlabel("Churn")
plt.ylabel("Satisfaction Score")

plt.show()
```



- Inactivity Impact on Churn

```python
# Customer Inactivity Impact on Churn (Correct)


# Clean data before plotting

df_clean = df.replace([np.inf, -np.inf], np.nan)

df_clean = df_clean.dropna(subset=["churn", "last_login_days_ago"])


# Create boxplot

plt.figure(figsize=(7,5))

sns.boxplot(x="churn", y="last_login_days_ago", data=df_clean)

plt.title("Customer Inactivity Impact on Churn")
plt.xlabel("Churn (0 = Retained, 1 = Churned)")
plt.ylabel("Days Since Last Login")

plt.show()
```



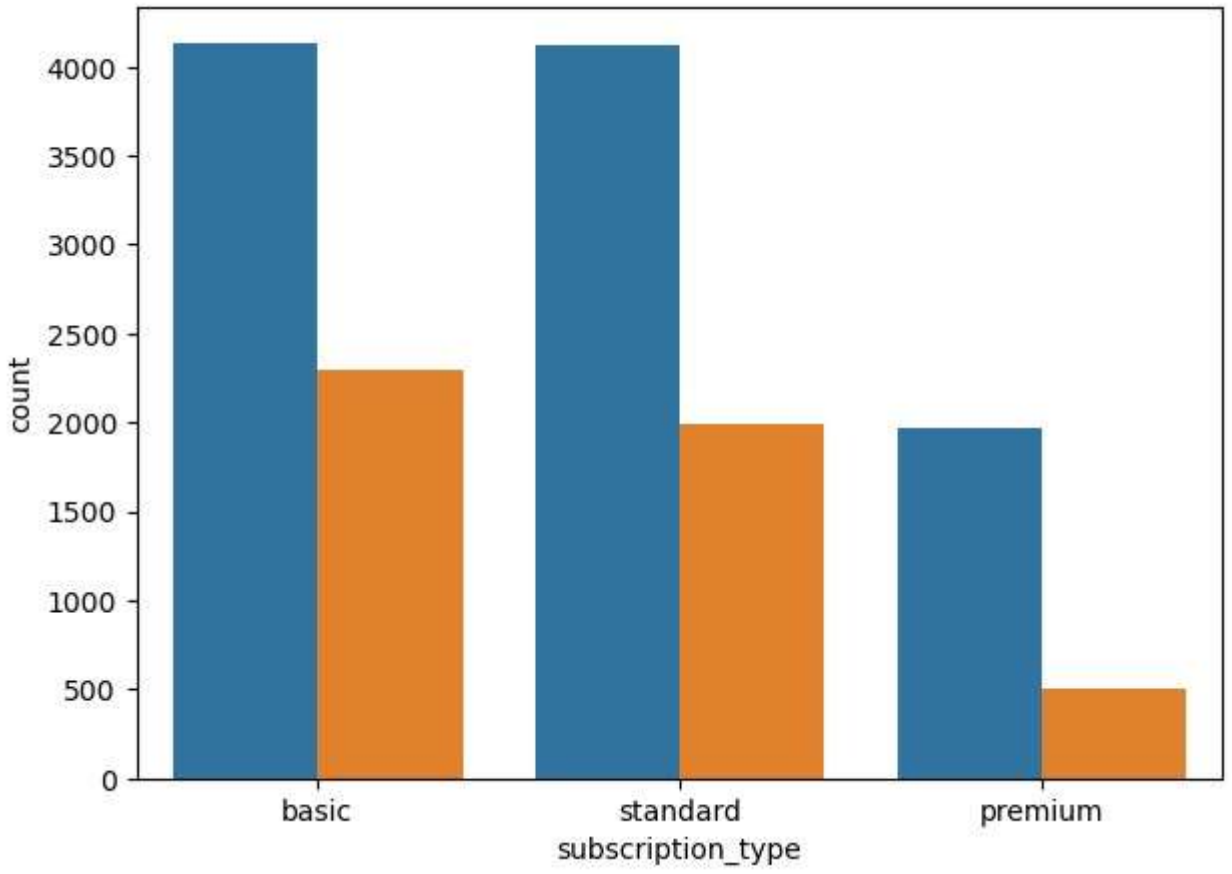
- Subscription Type Impact on Churn

```
#Categorical Feature Impact (Subscription Type)

# Table summary
subscription_impact = pd.crosstab(df["subscription_type"], df["churn"])

print("\nSubscription Type Impact on Churn:")
print(subscription_impact)
```

```
Subscription Type Impact on Churn:
churn                 0     1
subscription_type
basic              4134  2294
premium            1965   500
standard           4118  1989
```

- Important Billing Factor

```
# Visualization (important billing factor)
plt.figure(figsize=(7,5))

sns.countplot(x="subscription_type", hue="churn", data=df)

plt.title("Subscription Type Impact on Churn")
plt.xlabel("Subscription Type")
plt.ylabel("Number of Customers")

plt.show()
```



# Multivariate Analysis & Correlation

Goal:

- Analyze relationships between multiple features at the same time.

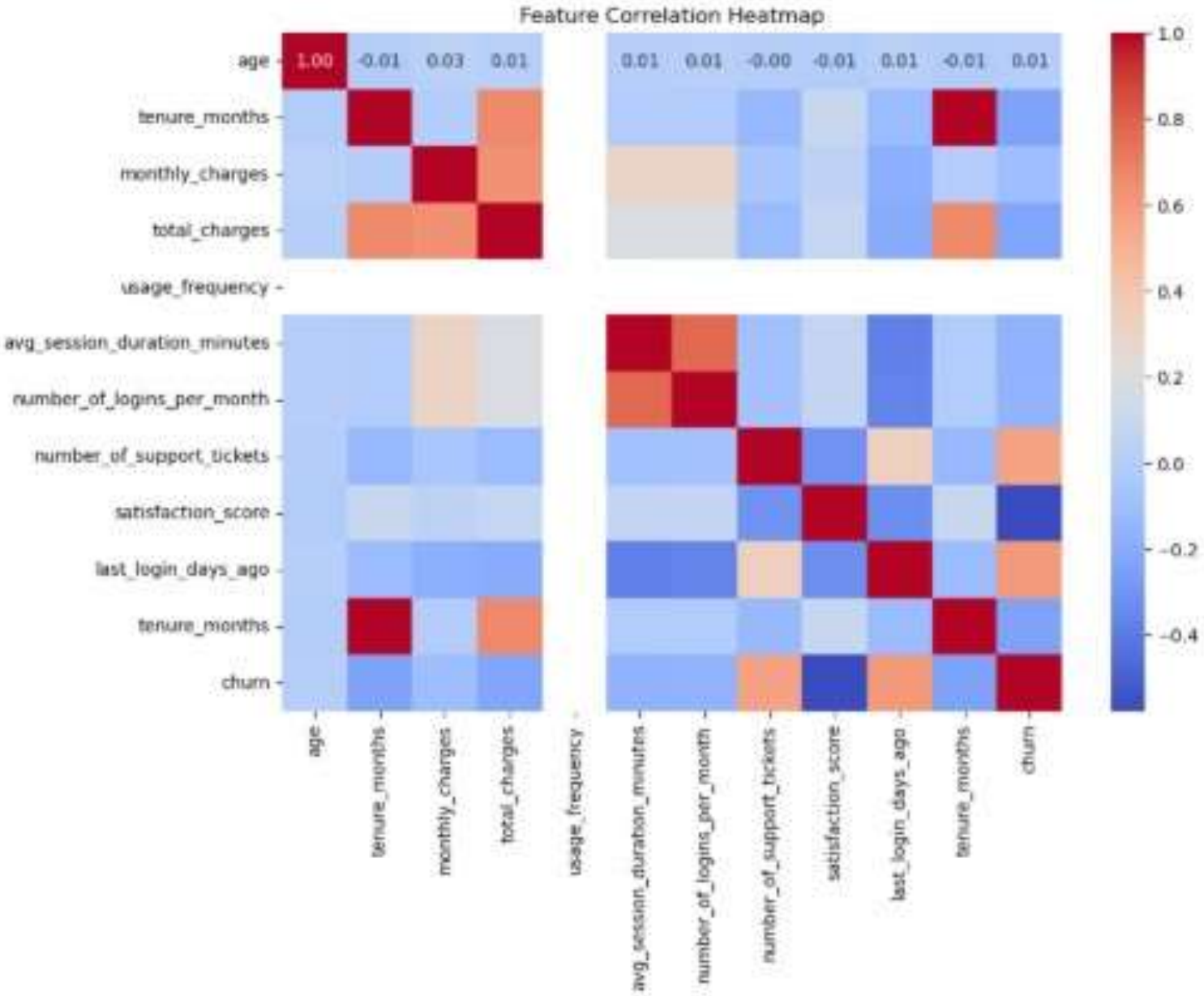
- Correlation helps identify which features are strongly related to churn and to each other.

- We focus on important numerical features from this churn dataset

- Creating Correlation Dataframe

```python
# Multivariate Analysis & Correlation (Correct)


# Define correct numerical features

numerical_features = [
    "age",
    "tenure_months",
    "monthly_charges",
    "total_charges",
    "usage_frequency",
    "avg_session_duration_minutes",
    "number_of_logins_per_month",
    "number_of_support_tickets",
    "satisfaction_score",
    "last_login_days_ago",
    "tenure_months",
    "churn"
]
```

```python
# Create correlation dataframe

corr_df = df[numerical_features]


# Show correlation matrix (table)

correlation_matrix = corr_df.corr()

print("Correlation Matrix:")
print(correlation_matrix)


##### Correlation Heatmap (Essential Graphic)


plt.figure(figsize=(10,7))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Feature Correlation Heatmap")

plt.show()
```


Feature Correlation Heatmap
age
tenure_months
monthly_charges
total_charges
usage_frequency
avg_session_duration_minutes
number_of_logins_per_month
number_of_support_tickets
satisfaction_score
last_login_days_ago
tenure_months
churn
1.00
-0.01
0.03
0.01
0.01
0.01
-0.00
-0.01
0.01
-0.01
0.01
1.0
0.8
0.6
0.4
0.2
0.0
-0.2
-0.4

- Correlation Matrix

```python
# Correlation values range from -1 to +1
# +1 → strong positive relationship
# -1 → strong negative relationship
# 0 → no relationship

correlation_matrix = corr_df.corr()

print("Correlation Matrix:")
print(correlation_matrix)
```

```
                             number_of_support_tickets  satisfaction_score  \
age                                           -0.002159           -0.005927
tenure_months                                 -0.131862            0.091599
monthly_charges                               -0.058143            0.042608
total_charges                                 -0.119793            0.080869
usage_frequency                                     NaN                 NaN
avg_session_duration_minutes                  -0.086456            0.075187
number_of_logins_per_month                    -0.083597            0.069643
number_of_support_tickets                      1.000000           -0.313925
satisfaction_score                            -0.313925            1.000000
last_login_days_ago                            0.337940           -0.325773
tenure_months                                 -0.131862            0.091599
churn                                          0.564679           -0.577549

                             last_login_days_ago  tenure_months     churn
age                                     0.013446      -0.009194  0.006893
tenure_months                          -0.119839       1.000000 -0.239026
monthly_charges                        -0.177398      -0.000998 -0.107328
total_charges                          -0.197224       0.662814 -0.226547
usage_frequency                              NaN            NaN       NaN
avg_session_duration_minutes           -0.385933      -0.012988 -0.167620
number_of_logins_per_month             -0.372261      -0.010783 -0.160164
number_of_support_tickets               0.337940      -0.131862  0.564679
satisfaction_score                     -0.325773       0.091599 -0.577549
last_login_days_ago                     1.000000      -0.119839  0.603591
tenure_months                          -0.119839       1.000000 -0.239026
churn                                   0.603591      -0.239026  1.000000
```

```
                             avg_session_duration_minute
age                                             0.007476
tenure_months                                  -0.012988
monthly_charges                                 0.304315
total_charges                                   0.188262
usage_frequency                                      NaN
avg_session_duration_minutes                    1.000000
number_of_logins_per_month                      0.776635
number_of_support_tickets                      -0.086456
satisfaction_score                              0.075187
last_login_days_ago                            -0.385933
tenure_months                                  -0.012988
churn                                          -0.167620

                             number_of_logins_per_month
age                                            0.008811
tenure_months                                 -0.010783
monthly_charges                                0.308904
total_charges                                  0.190737
usage_frequency                                     NaN
avg_session_duration_minutes                   0.776635
number_of_logins_per_month                     1.000000
number_of_support_tickets                     -0.083597
satisfaction_score                             0.069643
last_login_days_ago                           -0.372261
tenure_months                                 -0.010783
churn                                         -0.160164
```

```
Correlation Matrix:
                              age  tenure_months  monthly_charges
age                      1.000000      -0.009194         0.028310
tenure_months           -0.009194       1.000000        -0.000998
monthly_charges          0.028310      -0.000998         1.000000
total_charges            0.011781       0.662814         0.634164
usage_frequency               NaN            NaN              NaN
avg_session_duration_minutes  0.007470      -0.012988         0.304315
number_of_logins_per_month  0.008811      -0.010783         0.308904
number_of_support_tickets  -0.002159      -0.131862        -0.058143
satisfaction_score         -0.005927       0.091599         0.042608
last_login_days_ago         0.013446      -0.119839        -0.177398
tenure_months              -0.009194       1.000000        -0.000998
churn                       0.006893      -0.239026        -0.107328

                              total_charges  usage_frequency  \
age                                0.011781              NaN
tenure_months                      0.662814              NaN
monthly_charges                    0.634164              NaN
total_charges                      1.000000              NaN
usage_frequency                         NaN              NaN
avg_session_duration_minutes       0.188262              NaN
number_of_logins_per_month         0.190737              NaN
number_of_support_tickets         -0.119793              NaN
satisfaction_score                 0.080869              NaN
last_login_days_ago               -0.197224              NaN
tenure_months                      0.662814              NaN
churn                             -0.226547              NaN
```

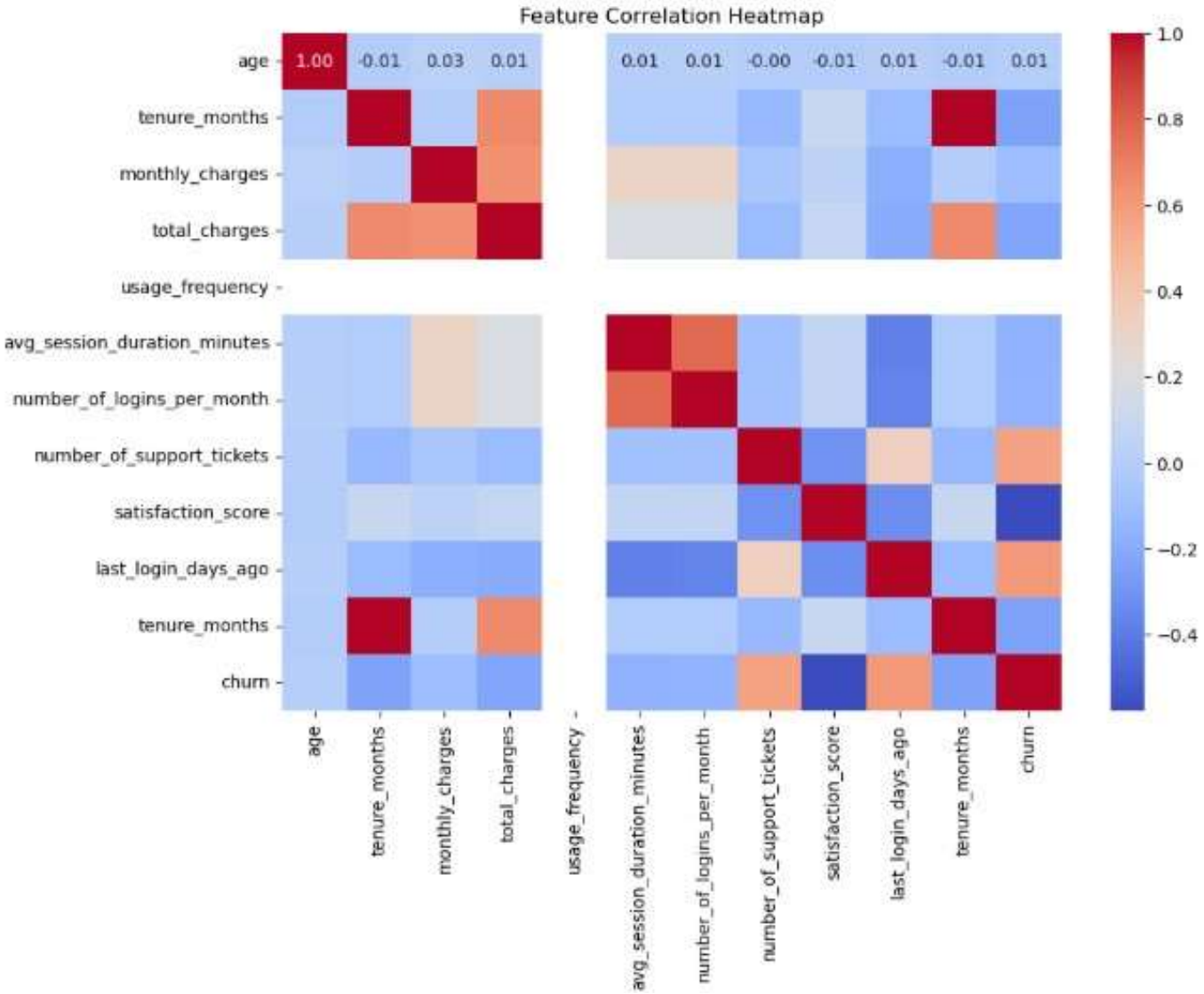
- Feature Correlation with Churn

```
#shows which features are most related to churn

churn_correlation = correlation_matrix["churn"].sort_values(ascending=False)

print("\nFeature Correlation with Churn:")
print(churn_correlation)
```

```
Feature Correlation with Churn:
churn                           1.000000
last_login_days_ago             0.603591
number_of_support_tickets       0.564679
age                             0.006893
monthly_charges                -0.107328
number_of_logins_per_month     -0.160164
avg_session_duration_minutes   -0.167620
total_charges                  -0.226547
tenure_months                  -0.239026
tenure_months                  -0.239026
satisfaction_score             -0.577549
usage_frequency                      NaN
Name: churn, dtype: float64
```

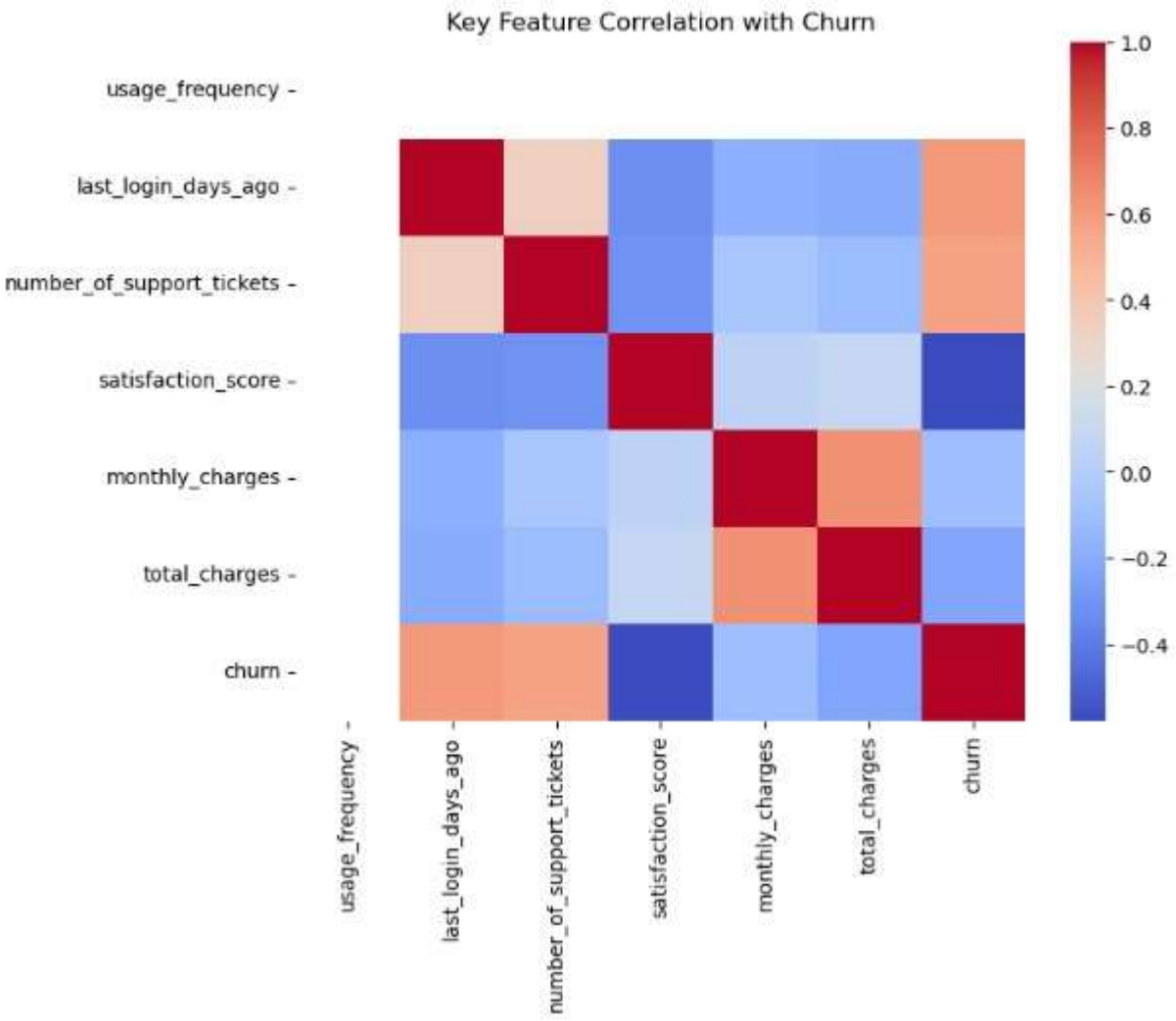
- Feature Correlation Heatmap

```python
# Heatmap helps visually identify strong relationships

plt.figure(figsize=(10,7))

sns.heatmap(
    correlation_matrix,
    annot=True,         # show correlation values
    cmap="coolwarm",    # color scheme
    fmt=".2f"           # show values with 2 decimal places
)

plt.title("Feature Correlation Heatmap")

plt.show()
```



- Key Feature Correlation with Churn

```python
# Automatically select important churn-related features

key_features = [
    "usage_frequency",
    "last_login_days_ago",
    "number_of_support_tickets",
    "satisfaction_score",
    "monthly_charges",
    "total_charges",
    "churn"
]

# Create heatmap

plt.figure(figsize=(8,6))

sns.heatmap(df[key_features].corr(), annot=True, cmap="coolwarm", fmt=".2f")

plt.title("Key Feature Correlation with Churn")

plt.show()
```



# Churn Risk Segmentation

Goal:

•Segment customers into different churn risk groups based on behavior.

•This helps businesses identify high-risk customers and take preventive action.

•Based on dataset logic, churn risk depends on:

•Satisfaction_score (low score = high risk)
•Usage_frequency (low usage = high risk)
•Last_login_days (high inactivity = high risk)

- Churn Risk Segmentation

```python
# Churn Risk Segmentation (Correct)


# Initialize risk score

df["risk_score"] = 0


# Add risk if satisfaction is low (<= 2)

df.loc[df["satisfaction_score"] <= 2, "risk_score"] += 1


# Add risk if usage frequency is low (below median)

df.loc[
    df["usage_frequency"] < df["usage_frequency"].median(),
    "risk_score"
] += 1


# Add risk if customer inactive for long time

df.loc[
    df["last_login_days_ago"] > df["last_login_days_ago"].median(),
    "risk_score"
] += 1


# Show sample output

print(df[[
    "satisfaction_score",
    "usage_frequency",
    "last_login_days_ago",
    "risk_score"
]].head())
```

| | satisfaction_score | usage_frequency | last_login_days_ago | risk_score |
|---|---|---|---|---|
| 0 | 5 | NaN | 4 | 0 |
| 1 | 3 | NaN | 13 | 0 |
| 2 | 5 | NaN | 54 | 1 |
| 3 | 3 | NaN | 6 | 0 |
| 4 | 5 | NaN | 59 | 1 |

- Churn Risk Segment Distribution

```python
# Convert risk_score into risk levels

def risk_category(score):

    if score == 0:
        return "Low Risk"

    elif score == 1:
        return "Medium Risk"

    else:
        return "High Risk"


# Apply function to create risk segment
df["churn_risk_segment"] = df["risk_score"].apply(risk_category)


# Check distribution of risk segments
print("\nChurn Risk Segment Distribution:")
print(df["churn_risk_segment"].value_counts())
```

```
Churn Risk Segment Distribution:
churn_risk_segment
Low Risk        6438
Medium Risk     6016
High Risk       2546
Name: count, dtype: int64
```

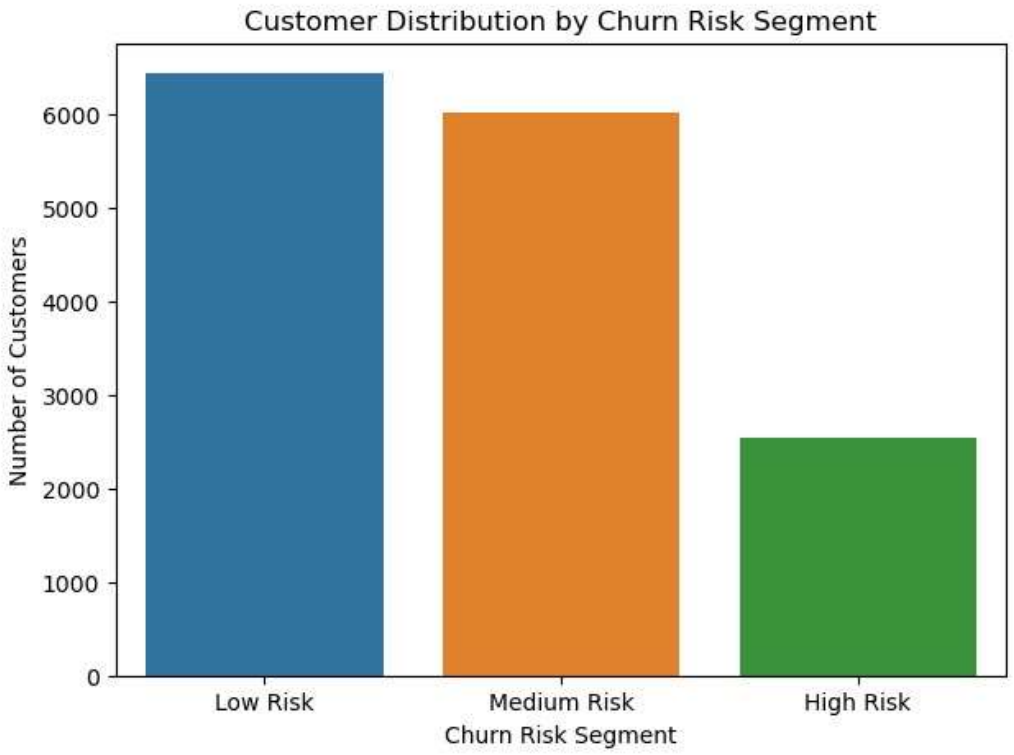
- Churn vs Risk Segment

```
#  Compare churn rate across risk segments (Important table)


risk_vs_churn = pd.crosstab(df["churn_risk_segment"], df["churn"])

print("\nChurn vs Risk Segment:")
print(risk_vs_churn)
```

```
Churn vs Risk Segment:
churn                   0     1
churn_risk_segment
High Risk             255  2291
Low Risk             6103   335
Medium Risk          3859  2157
```

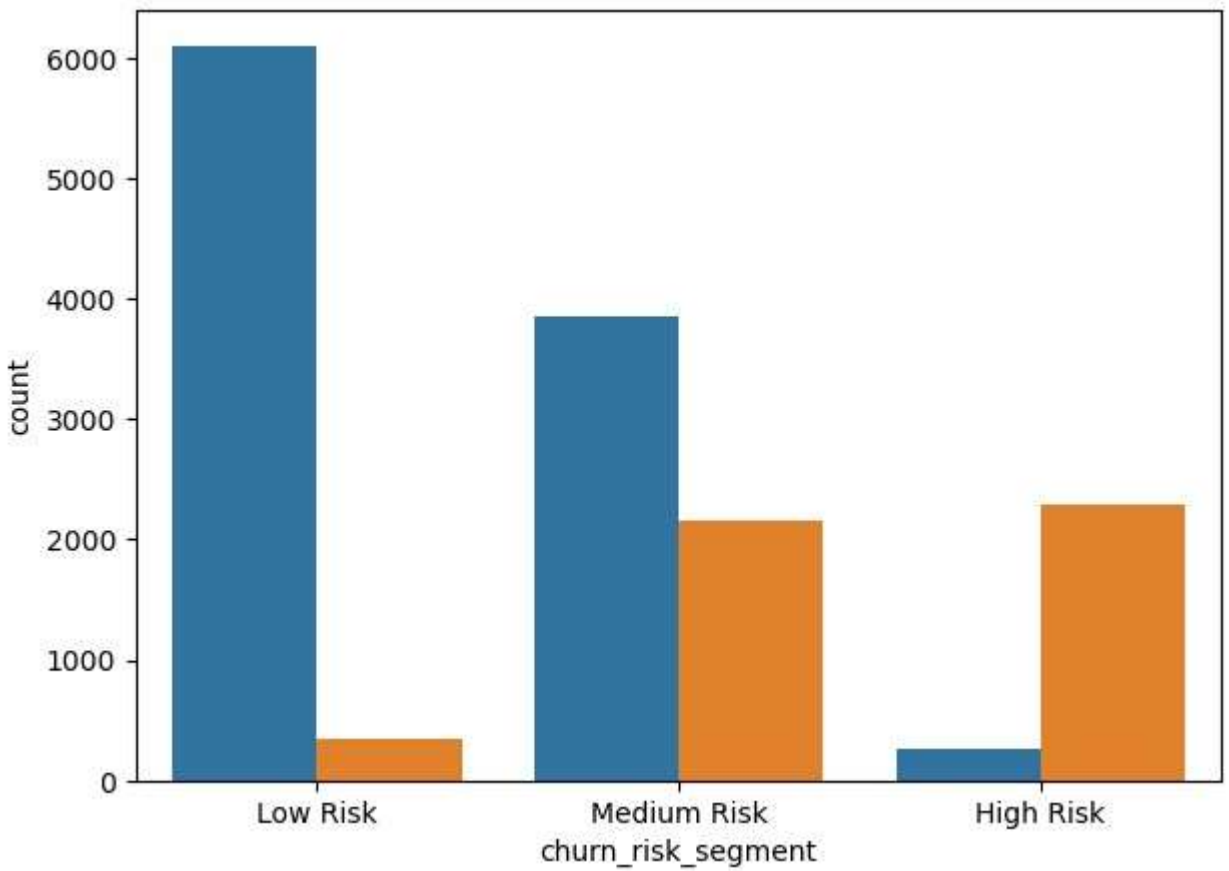
- Customer Distribution by Churn Risk Segment

```
# This graph shows number of customers in each risk category

plt.figure(figsize=(7,5))

sns.countplot(x="churn_risk_segment", data=df, order=["Low Risk", "Medium Risk", "High Risk"])

plt.title("Customer Distribution by Churn Risk Segment")
plt.xlabel("Churn Risk Segment")
plt.ylabel("Number of Customers")

plt.show()
```



- Churn Rate by Risk Segment

```
# This shows churn proportion within each risk group

plt.figure(figsize=(7,5))

sns.countplot(
    x="churn_risk_segment",
    hue="churn",
    data=df,
    order=["Low Risk", "Medium Risk", "High Risk"]
)

plt.title("Churn Rate by Risk Segment")
plt.xlabel("Churn Risk Segment")
plt.ylabel("Number of Customers")

plt.show()
```



# Business Questions
# &
# Answers

- How does customer satisfaction score affect churn probability?

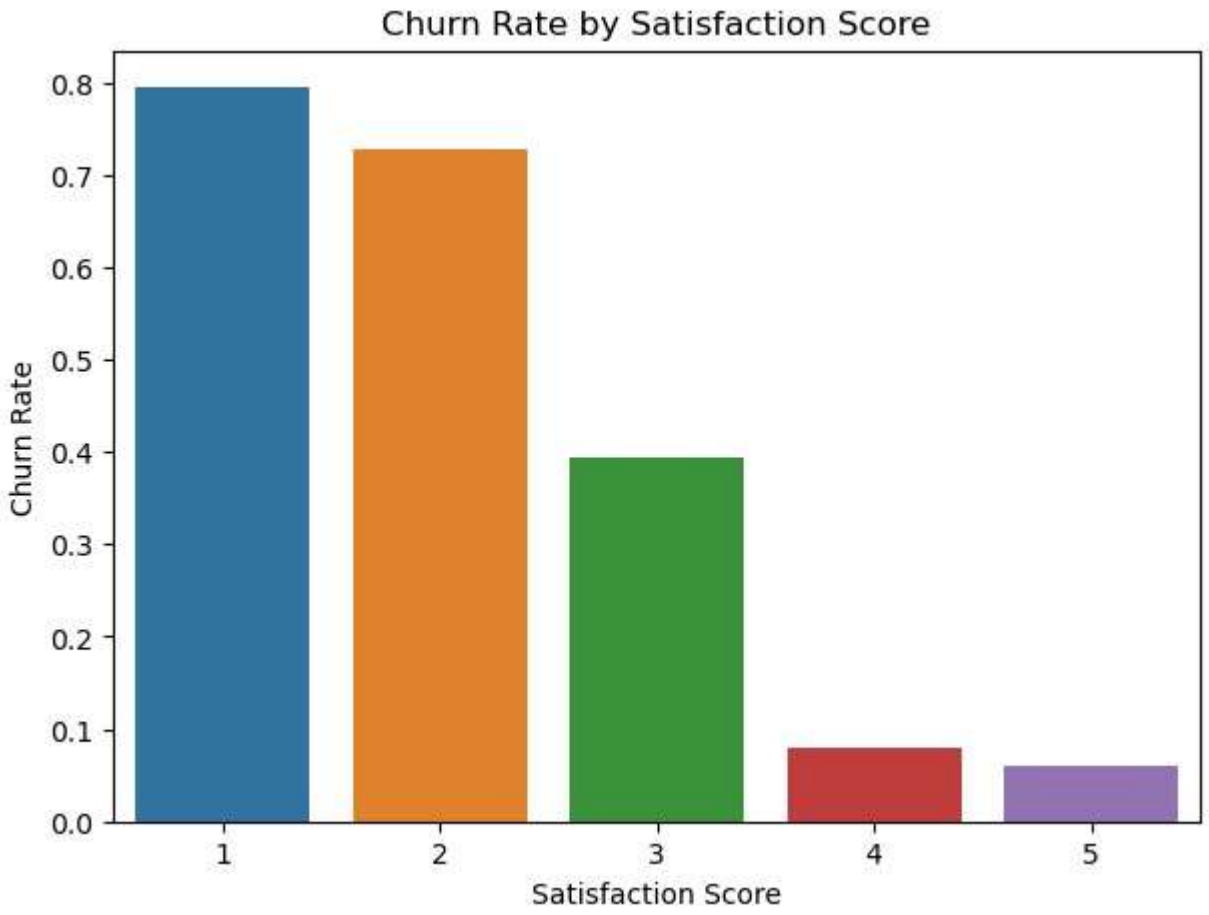
```python
# Group customers by satisfaction score and calculate churn rate

satisfaction_churn = df.groupby("satisfaction_score")["churn"].mean()

print("Churn Rate by Satisfaction Score:")
print(satisfaction_churn)


# Visualization to clearly see relationship

plt.figure(figsize=(7,5))

sns.barplot(x=satisfaction_churn.index, y=satisfaction_churn.values)

plt.title("Churn Rate by Satisfaction Score")
plt.xlabel("Satisfaction Score")
plt.ylabel("Churn Rate")

plt.show()
```

```
Churn Rate by Satisfaction Score:
satisfaction_score
1    0.794661
2    0.728011
3    0.393228
4    0.079301
5    0.059088
Name: churn, dtype: float64
```

- How does customer satisfaction score affect churn probability?

```python
# Group customers by satisfaction score and calculate churn rate

satisfaction_churn = df.groupby("satisfaction_score")["churn"].mean()

print("Churn Rate by Satisfaction Score:")
print(satisfaction_churn)


# Visualization to clearly see relationship

plt.figure(figsize=(7,5))

sns.barplot(x=satisfaction_churn.index, y=satisfaction_churn.values)

plt.title("Churn Rate by Satisfaction Score")
plt.xlabel("Satisfaction Score")
plt.ylabel("Churn Rate")

plt.show()
```

```
Churn Rate by Satisfaction Score:
satisfaction_score
1    0.794661
2    0.728011
3    0.393228
4    0.079301
5    0.059088
Name: churn, dtype: float64
```



- How does customer inactivity (last login days) impact churn risk?

```python
# Compare inactivity between retained and churned customers

inactivity_churn = df.groupby("churn")["last_login_days_ago"].mean()

print("Average Last Login Days by Churn:")
print(inactivity_churn)
```

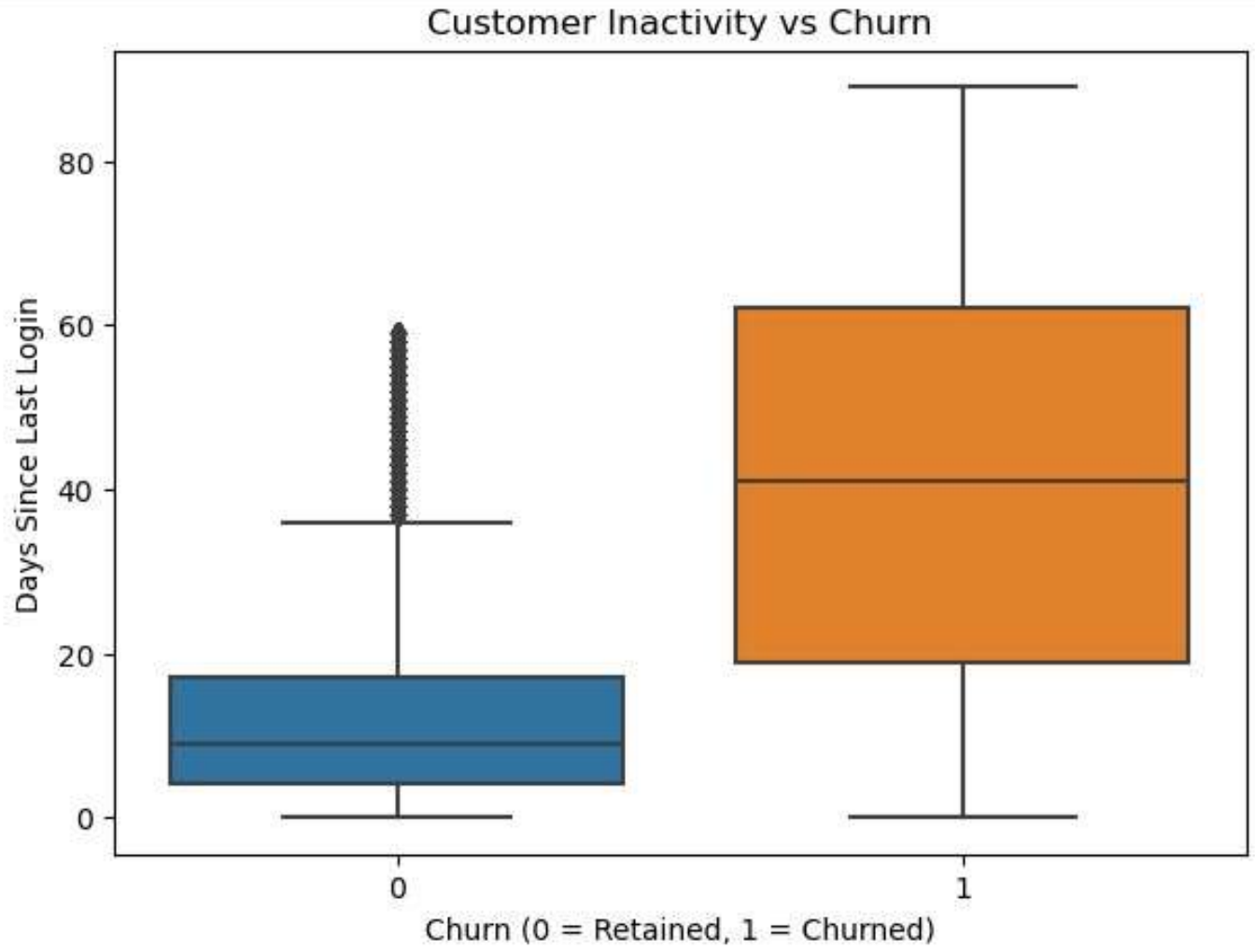
```
Average Last Login Days by Churn:
churn
0    12.546540
1    41.404976
Name: last_login_days_ago, dtype: float64
```

- How does customer inactivity (last login days) impact churn risk?

```python
# Compare inactivity between retained and churned customers

inactivity_churn = df.groupby("churn")["last_login_days_ago"].mean()

print("Average Last Login Days by Churn:")
print(inactivity_churn)
```

```
Average Last Login Days by Churn:
churn
0    12.546540
1    41.404976
Name: last_login_days_ago, dtype: float64
```

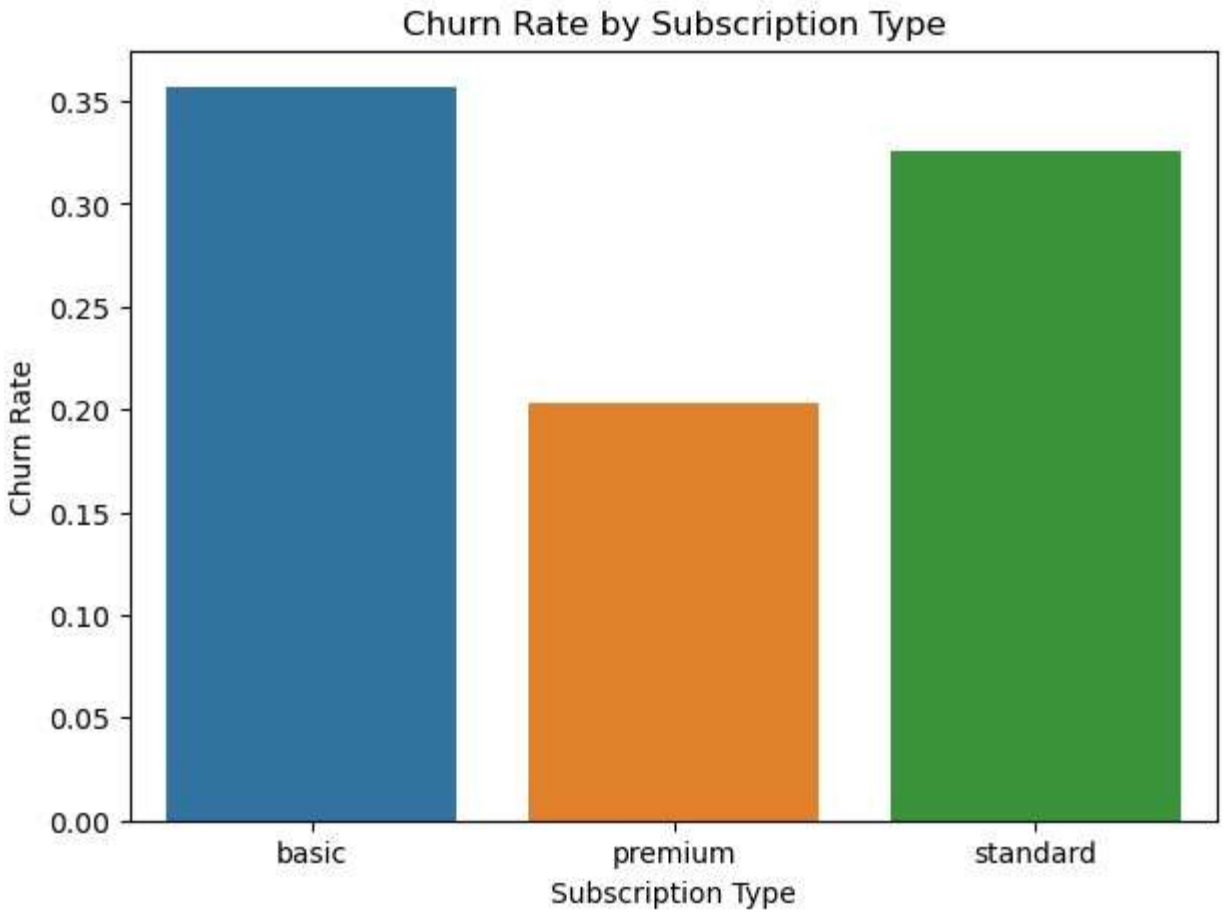
```python
# Clean data before plotting



df_plot = df.replace([np.inf, -np.inf], np.nan)

df_plot = df_plot.dropna(subset=["churn", "last_login_days_ago"])


# Create boxplot

plt.figure(figsize=(7,5))

sns.boxplot(x="churn", y="last_login_days_ago", data=df_plot)

plt.title("Customer Inactivity vs Churn")
plt.xlabel("Churn (0 = Retained, 1 = Churned)")
plt.ylabel("Days Since Last Login")

plt.show()
```



- Which subscription type has the highest churn rate?

```python
# Calculate churn rate by subscription type

subscription_churn = df.groupby("subscription_type")["churn"].mean()

print("Churn Rate by Subscription Type:")
print(subscription_churn)


# Visualization

plt.figure(figsize=(7,5))

sns.barplot(x=subscription_churn.index, y=subscription_churn.values)

plt.title("Churn Rate by Subscription Type")
plt.xlabel("Subscription Type")
plt.ylabel("Churn Rate")

plt.show()
```

```
Churn Rate by Subscription Type:
subscription_type
basic       0.356876
premium     0.202840
standard    0.325692
Name: churn, dtype: float64
```

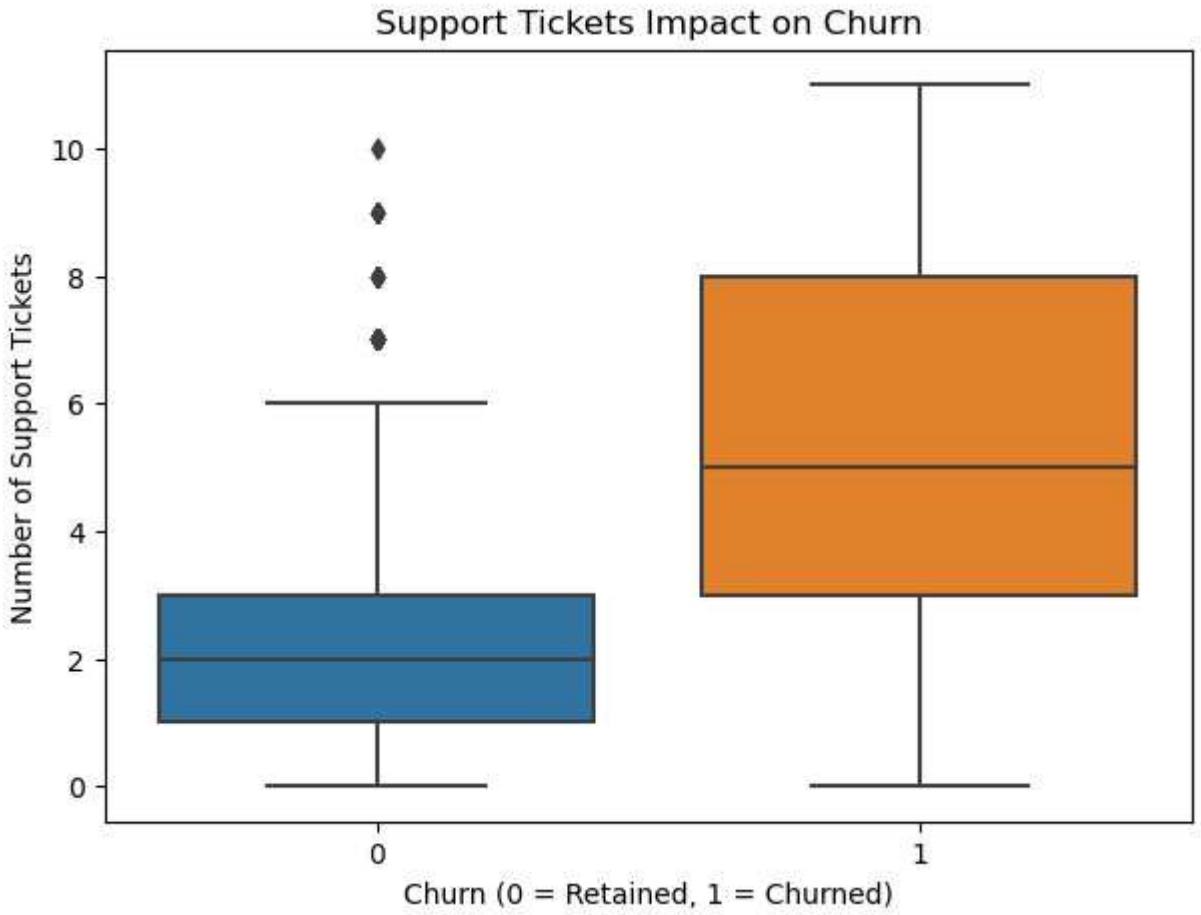


- Do customers with more support tickets have higher churn?

```python
# Support Tickets Impact on Churn (Correct)


# Compare average support tickets for retained vs churned customers

support_churn = df.groupby("churn")["number_of_support_tickets"].mean()

print("Average Support Tickets by Churn:")
print(support_churn)
```

```python
# Clean data before plotting


df_plot = df.replace([np.inf, -np.inf], np.nan)

df_plot = df_plot.dropna(subset=["churn", "number_of_support_tickets"])


# Create boxplot

plt.figure(figsize=(7,5))

sns.boxplot(
    x="churn",
    y="number_of_support_tickets",
    data=df_plot
)

plt.title("Support Tickets Impact on Churn")
plt.xlabel("Churn (0 = Retained, 1 = Churned)")
plt.ylabel("Number of Support Tickets")

plt.show()
```

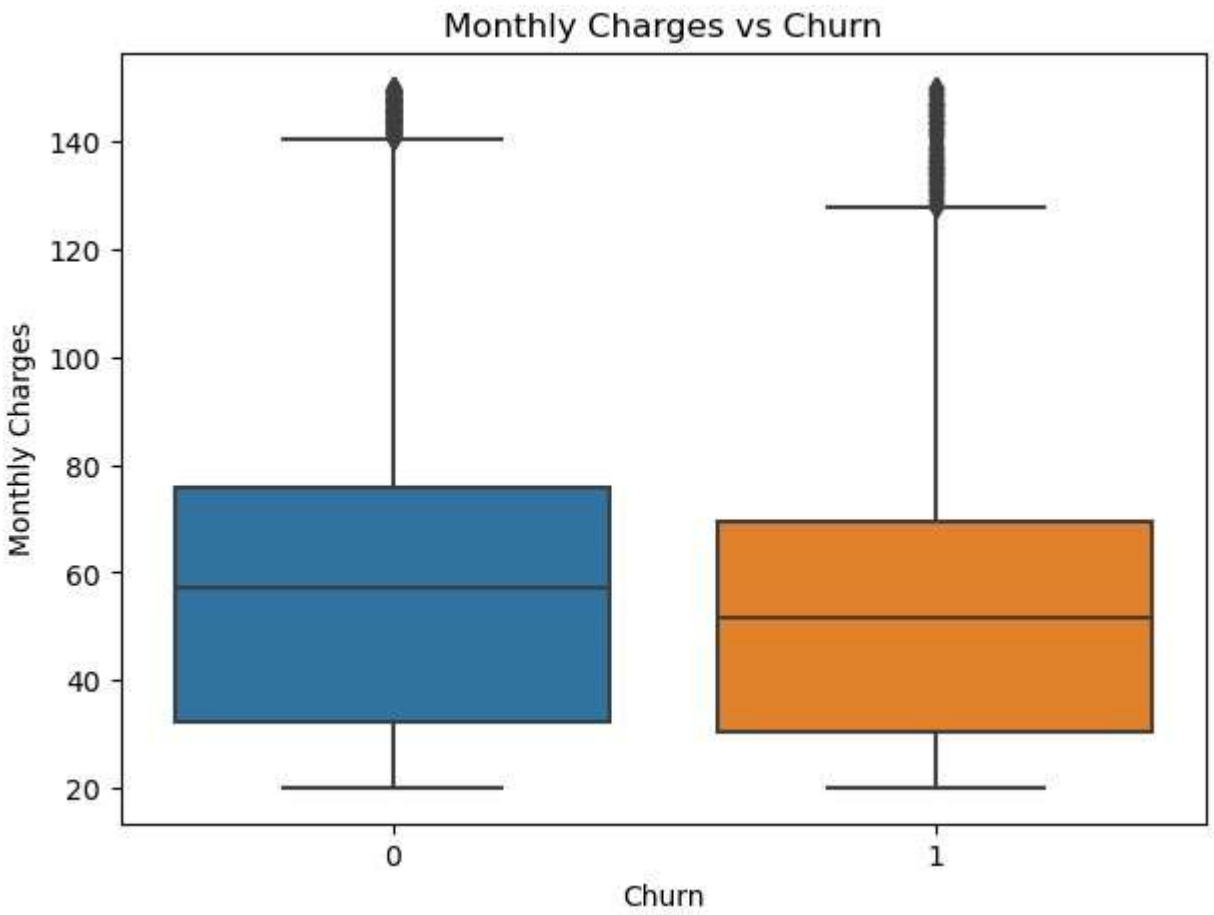
```
Average Support Tickets by Churn:
churn
0    1.919154
1    5.080284
Name: number_of_support_tickets, dtype: float64
```



- How do monthly charges influence churn?

```python
# Compare average monthly charges

charges_churn = df.groupby("churn")["monthly_charges"].mean()

print("Average Monthly Charges by Churn:")
print(charges_churn)


# Visualization

plt.figure(figsize=(7,5))

sns.boxplot(x="churn", y="monthly_charges", data=df)

plt.title("Monthly Charges vs Churn")
plt.xlabel("Churn")
plt.ylabel("Monthly Charges")

plt.show()
```

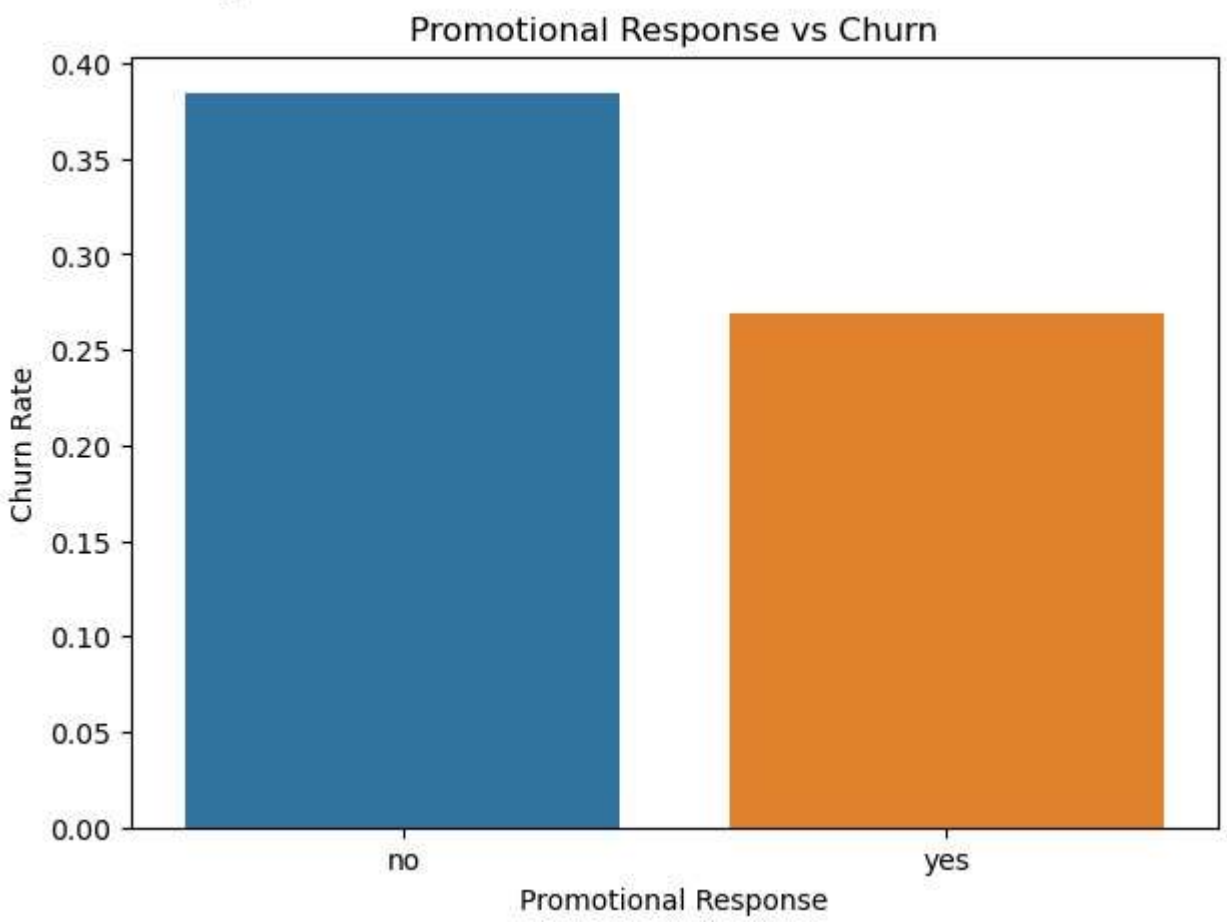
```
Average Monthly Charges by Churn:
churn
0    61.469184
1    53.870366
Name: monthly_charges, dtype: float64
```



- Do promotions and discounts improve retention?

```python
# Calculate churn rate based on promotional response

promo_churn = df.groupby("promotional_response")["churn"].mean()

print("Churn Rate by Promotional Response:")
print(promo_churn)


# Visualization

plt.figure(figsize=(7,5))

sns.barplot(x=promo_churn.index, y=promo_churn.values)

plt.title("Promotional Response vs Churn")
plt.xlabel("Promotional Response")
plt.ylabel("Churn Rate")

plt.show()
```

```
Churn Rate by Promotional Response:
promotional_response
no     0.384308
yes    0.268646
Name: churn, dtype: float64
```



# Key Insights & Patterns

## 1.Customer Satisfaction Drives Churn

Customer satisfaction score has a strong relationship with churn. Customers with lower satisfaction levels are significantly more likely to leave, highlighting the importance of improving service quality and overall customer experience.

## 2.Low Engagement Increases Churn Risk

Customers who use the platform less frequently or spend shorter time per session show higher churn rates, indicating that disengaged users are more likely to stop using the service.

## 3.Customer Inactivity Signals Potential Churn

Long periods without login activity often indicate declining interest. Inactive customers are more likely to churn, making inactivity an early warning indicator.

## 4.Frequent Support Tickets Indicate Dissatisfaction

Customers who raise a higher number of support tickets tend to churn more often, suggesting that unresolved issues or poor support experience contribute to customer loss.

## 5.Long-Term and High-Value Customers Are More Loyal

Customers with longer tenure and higher total spending generally show lower churn rates, indicating stronger loyalty and higher lifetime value.

## 6.Subscription and Billing Stability Improves Retention

Customers with stable subscription plans and consistent payment behavior tend to stay longer, reflecting satisfaction with pricing and perceived value.

## 7.Promotions and Discounts Support Retention

Customers who respond to promotional offers or use discounts are less likely to churn, showing that targeted incentives can positively influence retention.

## 8.Behavioral Factors Impact Churn More Than Demographics

Usage patterns, satisfaction levels, inactivity, and support interactions influence churn more strongly than demographic characteristics such as age or gender.

## 9.Risk-Based Segmentation Identifies At-Risk Customers

Customers categorized in the high-risk churn segment show significantly higher actual churn rates, validating the effectiveness of churn risk segmentation.

## 10.Churn Results from Multiple Behavioral Signals

Customer churn is rarely caused by a single factor. Instead, it results from a combination of behavioral indicators such as low engagement, poor satisfaction, inactivity, and frequent support issues.

# Business Recommendations

**1.Enhance Customer Satisfaction**

Customer satisfaction is the most influential factor in churn. Businesses should prioritize improving service quality, resolving issues quickly, and gathering regular feedback to strengthen the overall customer experience.

**2.Boost Customer Engagement**

Low engagement is closely linked to higher churn risk. Companies should encourage consistent usage through personalized notifications, feature recommendations, and targeted engagement campaigns.

**3.Track and Re-engage Inactive Customers**

Customer inactivity often signals potential churn. Identifying inactive users early and re-engaging them through reminders, personalized messages, or special offers can help bring them back to the platform.

**4.Improve Customer Support Effectiveness**

A high number of support tickets often reflects unresolved problems. Faster response times, efficient issue resolution, and improved support quality can significantly reduce churn.

**5.Offer Targeted Promotions to At-Risk Customers**

Promotions and discounts can positively influence retention. Data-driven strategies should be used to identify high-risk customers and provide personalized incentives to keep them engaged.

**6.Prioritize High-Value Customer Retention**

Customers with longer tenure and higher spending contribute more to long-term revenue. Loyalty programs, exclusive benefits, and personalized offers can help retain these valuable customers and increase their lifetime value.

# Summary & Conclusion

# Summary

This project conducted a comprehensive **Exploratory Data Analysis (EDA)** on a customer behavior and churn dataset from a subscription-based digital service. The main objective was to analyze customer behavior, identify the key drivers of churn, and generate meaningful business insights.

The analysis involved several steps, including **data cleaning, feature engineering, univariate analysis, behavioral analysis, billing and subscription analysis, multivariate correlation analysis, and churn risk segmentation**.

Key customer behavior indicators such as **satisfaction score, usage frequency, inactivity level, support interactions, and spending patterns** were examined to understand their influence on customer churn and overall engagement.

# Conclusion

Customer churn is a significant challenge for subscription-based businesses, but it can be effectively addressed through **data-driven analysis and proactive strategies**.
This project shows how **Exploratory Data Analysis** helps uncover hidden patterns in customer behavior, identify churn risk factors, and support informed business decisions.
By utilizing these behavioral insights, organizations can **enhance customer experience, improve retention strategies, and drive long-term sustainable growth**.

# THE END